

Year Up United Data Analyst Training Academy: Advanced Reports with Power BI

Week 11 Student Guide

This courseware is to be distributed to accompany instructor led training.

Table of Contents

<i>Week 11 Learning Objectives</i>	5
<i>Week 11 Learning Outcomes</i>	6
MODULE 1 ADVANCED DAX CONCEPTS	7
SECTION 1.A FILTERING	8
1.A.1 <i>Filtering</i>	9
1.A.2 <i>Implicit Filtering</i>	10
1.A.3 <i>Implicit Filtering Example</i>	11
1.A.4 <i>Explicit Filtering</i>	12
1.A.5 <i>CALCULATE() function</i>	13
1.A.6 <i>FILTER() function</i>	14
1.A.7 <i>FILTER() function example</i>	15
1.A.8 <i>ALL() Function</i>	16
SECTION 1.B CALCULATED TABLES	17
1.B.1 <i>Calculated Tables</i>	18
1.B.2 <i>Calculated Table Example</i>	19
SECTION 1.C ITERATORS	20
1.C.1 <i>Iterators</i>	21
1.C.2 <i>Evaluation Context</i>	22
1.C.3 <i>Common Iterator Functions</i>	23
1.C.4 <i>Nesting Iterators</i>	24
SECTION 1.D ROW LEVEL SECURITY	25
1.D.1 <i>Row Level Security</i>	26
1.D.2 <i>Defining RLS</i>	27
1.D.3 <i>Testing RLS</i>	28
1.D.4 <i>Users assigned in Power BI Service</i>	29
MODULE 2 DATE AND TIME INTELLIGENCE	30
SECTION 2.A DATES AND DATE TABLES	31
2.A.1 <i>Date Dimensions</i>	32
2.A.2 <i>Auto date/time in Power BI Desktop</i>	33
2.A.3 <i>Auto date/time limitations</i>	34
2.A.4 <i>Disabling Auto date/time</i>	35
2.A.5 <i>Date Tables</i>	36
2.A.6 <i>Holiday Fields</i>	37
2.A.7 <i>Time Tables</i>	38
2.A.8 <i>Creating Date Tables</i>	39
2.A.9 <i>Making a Date Table</i>	40
2.A.10 <i>Multiple Relationships and Date Tables</i>	41
2.A.11 <i>Avoiding Ambiguous Relationships</i>	42
2.A.12 <i>Role Playing Tables</i>	43
SECTION 2.B DAX TIME INTELLIGENCE FUNCTIONS	44
2.B.1 <i>Periodized Analysis: The Hard Way</i>	45
2.B.2 <i>Time Intelligence: The Easy Way</i>	46
2.B.3 <i>Time Intelligence Examples</i>	47

MODULE 3 PRACTICAL MODELING TECHNIQUES..... 48**SECTION 3.A MORE POWER QUERY CONCEPTS 49**

3.A.1	<i>Other Transformation Functions</i>	50
3.A.2	<i>Transpose</i>	51
3.A.3	<i>Pivot & Unpivot</i>	52
3.A.4	<i>Unpivot</i>	53
3.A.5	<i>Split Columns</i>	54
3.A.6	<i>Fill</i>	55
3.A.7	<i>Pivot</i>	56

SECTION 3.B COMBINING DATA SOURCES 57

3.B.1	<i>Folder Import</i>	58
3.B.2	<i>Combine</i>	59
3.B.3	<i>Custom Functions & Parameters</i>	60

MODULE 4 ADVANCED ANALYTICS WITH POWER BI 61**SECTION 4.A ADVANCED ANALYTICS FEATURES 62**

4.A.1	<i>Overview of Advanced Analytics</i>	63
4.A.2	<i>Key influencers visualization</i>	64
4.A.3	<i>Decomposition tree visualization</i>	65
4.A.4	<i>Quick Insights (Power BI Service)</i>	66
4.A.5	<i>Types of Quick Insights</i>	67
4.A.6	<i>Using Copilot with Power BI</i>	68
4.A.7	<i>Other data exploration and analysis tools</i>	69

MODULE 5 SHARING POWER BI REPORTS..... 70**SECTION 5.A POWER BI SERVICE 71**

5.A.1	<i>Power BI Service</i>	72
5.A.2	<i>Collaborate Using Workspaces</i>	73
5.A.3	<i>Publishing to a Workspace</i>	74
5.A.4	<i>Tailoring Report for Mobile Sharing</i>	75
5.A.5	<i>Sharing from within Power BI Service</i>	76
5.A.6	<i>Export to PowerPoint</i>	77
5.A.7	<i>Analyze in Excel</i>	78

SECTION 5.B OTHER REPORT EXPORT OPTIONS 79

5.B.1	<i>Exporting to PDF</i>	80
5.B.2	<i>Exporting a Template</i>	81

SECTION 5.C OTHER POWER BI SERVICE FEATURES 82

5.C.1	<i>Beyond Report Sharing</i>	83
5.C.2	<i>Power BI Service Basics</i>	84
5.C.3	<i>Dataflows</i>	85
5.C.4	<i>Dashboards</i>	86
5.C.5	<i>Workspaces</i>	87
5.C.6	<i>Admin Role for Workspaces</i>	88
5.C.7	<i>Apps in Power BI Service</i>	89
5.C.8	<i>Deployment pipelines</i>	90
5.C.9	<i>Data Hub</i>	91

MODULE 6 DATA GOVERNANCE 92

SECTION 6.A DATA GOVERNANCE BASICS 93

- 6.A.1 *Introducing Data Governance* 94
- 6.A.2 *Data Culture* 95
- 6.A.3 *Key Concepts in Data Governance* 96
- 6.A.4 *Data Quality* 97
- 6.A.5 *Data Lineage* 98
- 6.A.6 *Data Stewardship* 99
- 6.A.7 *Data Security* 100
- 6.A.8 *Week 11 Wrap-Up* 101

Week 11 Learning Objectives

- This week you will learn:
 - ◇ Explicit DAX filtering: CALCULATE(), FILTER(), and ALL()
 - ◇ Calculated tables: use cases and examples functions
 - ◇ DAX iterators: evaluation context (row context inside an iterator vs. filter context), common X-functions, and guidelines for nesting iterators
 - ◇ Row Level Security: defining DAX-based roles, applying row filters, testing RLS within Power BI Desktop, and role assignment in Power BI service
 - ◇ Date dimensions: the uses of date tables (custom date fields, fiscal calendars) and best practices for date table structure
 - ◇ DAX time intelligence functions: requirements (marked date table, contiguous date range), categories of functions, and common patterns using DATESYTD(), TOTALYTD(), and SAMEPERIODLASTYEAR()
 - ◇ Advanced Power Query transformations: Transpose, Split Columns, Fill (up/down), Pivot (rows to columns), and Unpivot (columns to rows)
 - ◇ Combining data sources in Power Query: folder import, Combine, and custom functions and parameters in M
 - ◇ Power BI Service: publishing to a workspace, sharing and permission options, mobile layout, and integration with Teams and PowerPoint
 - ◇ AI and advanced analytics features built into Power BI: clustering, decomposition trees, key influencers visualization, and Quick Insights
 - ◇ Power BI Service deployment tools: deployment pipelines, Dataflows, and the OneLake Data Hub
 - ◇ Data governance concepts: data quality, data lineage, data stewardship, and data security

Week 11 Learning Outcomes

- By the end of this week, you will be able to:
 - ◇ Write a DAX measure using CALCULATE() and ALL() to override the implicit filter context for an expression
 - ◇ Write a FILTER()-based expression that applies criteria spanning multiple columns
 - ◇ Create a custom date table in Power BI, mark it as a date table, and use it with Time Intelligence functions such as DATESYTD() and TOTALYTD()
 - ◇ Apply Unpivot in Power Query to transform a "wide" spreadsheet into a normalized "tall" format, and Pivot to reshape data from “tall” to “wide”, to reshape data to be more suitable for analysis
 - ◇ Publish a report to a Power BI Service workspace and share it with at least one other user with appropriate permissions

Module 1

Advanced DAX Concepts

Section 1.A

Filtering

1.A.1 Filtering

- As you have already seen, filters are used to filter a subset of values from the data that are used in calculation
 - ◇ You might filter values based on a dimension's attribute
 - * For example filtering Sales Amount by Product Category
 - ◇ You might "drill down" to see how a large summary is split up
 - * Such as expanding Category and looking at the Subcategories or Products below it
 - ◇ You might perform Period-Based comparisons
 - * To compute how this year is doing compared to last year
 - * To determine year-to-date or month-to-date sales

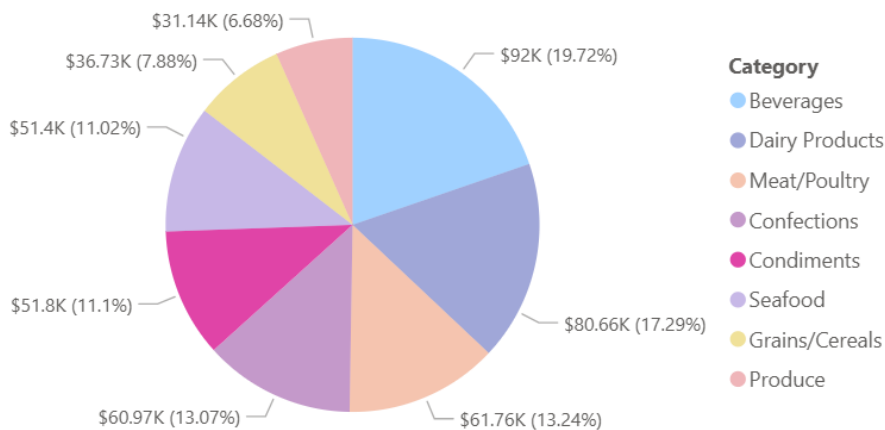
1.A.2 Implicit Filtering

- Implicit filtering is filtering applied by the user or the layout of the report
 - ◇ When you add a field to a bar chart, the DAX engine is implicitly filtering by that field for each bar
 - * This means that each bar has its own implicit filter context
 - ◇ When you click on a pie slice within a pie chart, the DAX engine changes the filter context so that all the other visualizations on the page only see the data represented by that slice of the pie

1.A.3 Implicit Filtering Example

- We can see implicit filtering at two places in the below visual
 - ◇ In the table, at the intersection of Argentina and Sale Amount, we see that SUM(Sale Amount) has been filtered to show only the sales that happened in Argentina
 - * The value of \$480 represents Argentina, no other country
 - ◇ Furthermore, the Condiments slice has been selected in the pie chart (with other slices showing up semi-transparent to indicate they have been filtered out)
 - * Filtering on the pie chart means that the table visualization also is only showing Condiment sales
 - * This means that the \$480 of Sales Amount in Argentina is specifically the sales in Argentina for Condiments

Total Sales by Category



Order Ship Country	Sum of Sales Amount
Argentina	\$480
Austria	\$5,980.25
Belgium	\$1,732.7
Brazil	\$4,202
Canada	\$1,542.5
Denmark	\$574.6
Finland	\$1,120
France	\$3,638
Germany	\$7,479.4
Ireland	\$1,297
Italy	\$540.25
Mexico	\$175.05
Poland	\$627
Portugal	\$3,236.9
Total	\$51,797.3

1.A.4 Explicit Filtering

- Explicit filtering is filtering applied by the DAX formula itself
- Explicit filtering will override implicit filtering on a column by column basis
- You can specify filters in DAX expressions with these functions:
 - ◇ Use `CALCULATE()` to change the filter context for an expression
 - ◇ Use `FILTER()` to apply advanced filters based on row-level conditions
 - ◇ `ALL()` lets you undo implicit filters

1.A.5 CALCULATE() function

- The CALCULATE() function is the fastest way to explicitly filter data in DAX

◇ CALCULATE() requires an expression and a filter condition

◇ Example

```
Sum Green Products Sold = CALCULATE(  
    SUM(Sales[Total]),  
    [Color] = "Green"  
)
```

- Using the CALCULATE() function by itself is very fast

◇ This is because it uses the fact that the columnar database has a single column's data stored together to its advantage

◇ Each filter argument in CALCULATE() must reference a single column using a simple comparison

- Example

◇ The following will work:

```
Average Freight for Products Greater than Ten = CALCULATE(  
    AVERAGE(Sales[Freight Amount]),  
    Sales[Sales Amount] > 10  
)
```

◇ Multiple columns in the condition will **not** work:

```
Average Freight for Products Greater than Ten = CALCULATE(  
    AVERAGE(Sales[Freight Amount]),  
    Sales[Unit Price] * ( 1 - [Discount Percent]) > 10  
)
```

1.A.6 FILTER() function

- The filter function can be used when you need to consider multiple columns in your filter criteria
- The FILTER() function takes two parameters
 - ◇ A table value that represents the input data
 - ◇ An expression that will be evaluated for each row in the input data
- The result is a table object, which means the FILTER() function can be used anywhere that expects a table
 - ◇ The return table will contain each row from the input where the expression evaluated to true
- The result of the FILTER() function is passed to another function for operations such as custom aggregations
- FILTER() can be used inside of CALCULATE()
 - ◇ This can be an extremely useful combination
 - ◇ But when you do this, it significantly slows down the calculation

1.A.7 FILTER() function example

- Examples:

The following example is a simple example of the FILTER() function

```
Products = FILTER(  
    'Products',  
    'Products'[BrandName] = "GoodForYou"  
)
```

In this example, we're demonstrating that FILTER() can utilize multiple columns in the condition applied

```
HighMarginProducts = FILTER(  
    'Products',  
    AND('Products'[BrandName] = "GoodForYou",  
        'Products'[UnitPrice] > 'Products'[UnitCost] * 2)  
)
```

1.A.8 ALL() Function

- The ALL() function is a way to clear any implicit filters
 - ◇ Can be applied to a specific column
 - ◇ Can be applied to an entire table
- This is useful in situations where you want to perform ratio between the filtered value and the original whole value

- ◇ Example

```
SalesPercent = DIVIDE(  
    SUMX(  
        Sales,  
        Sales[Quantity] * Sales[NetPrice]  
    ),  
    SUMX(  
        ALL(Sales),  
        Sales[Quantity] * Sales[NetPrice]  
    )  
)
```


Section 1.B

Calculated Tables

1.B.1 Calculated Tables

- Calculated tables are the third of four types of calculations that can be added to a model with DAX
- A calculated table must be a formula that returns a table data type
 - ◇ ... such as the FILTER() function, for example
- Using calculated tables is sometimes easier than repeating the same FILTER() expression in different calculations
- Can be used to create "role playing" tables
 - ◇ Used to overcome the limitation of having only a single active relationship between tables
 - ◇ Will be discussed in detail when we cover Date / Time

1.B.2 Calculated Table Example

- Examples

- ◇ Creates a table named Global Sales that represents the combined data of the NorthAmericanSales and EuropeanSales tables:

```
Global Sales = UNION(NorthAmericanSales, EuropeanSales)
```

- ◇ Creates a table named Product Categories that combines the Categories and Subcategories tables through an inner join

```
Product Categories = NATURALINNERJOIN(Categories, Subcategories)
```

- * This could be used as an alternative way to flatten data from multiple tables into a single table, and might be preferred when more than 1 or 2 columns are being brought over

- ◇ Creates a table named Order Status that "hard codes" text descriptions of numeric OrderStatusId values

```
Order Status = DATATABLE(  
    "OrderStatusId", Integer, "Status", String,  
    {{1, "In Cart"},  
     {2, "Placed"},  
     {3, "Acknowledged"},  
     {4, "Shipped"},  
     {5, "Delivered"},  
     {6, "Cancelled"}}  
)
```

Section 1.C

Iterators

1.C.1 Iterators

- An iterator is a special class of functions
- Many of the built-in iterator functions end with the letter X
- An iterator accepts a table value and an expression
- Some return table values, others return scalar values
 - ◇ FILTER() is an example of an iterator that returns a table value
 - ◇ SUMX() is an example of an iterator that returns a scalar value
- Example
 - ◇ In this example, Sales[Net Price] * Sales[Quantity] is going to be evaluated for each row in Sales, then each one of the return values will be added together to produce a sum

```
Total Sales = SUMX( Sales, Sales[Net Price] * Sales[Quantity] )
```

Table Value

Expression

Iterator

1.C.2 Evaluation Context

- Evaluation context is the combination of row and filter contexts in which a formula is evaluated
- Row Context
 - ◇ Limits the current row in an iterator
 - ◇ Used in calculated columns
 - ◇ Used in iterators
 - ◇ Does not propagate filters
 - ◇ Uses RELATED() or RELATEDTABLE() to access data in other tables
- Filter Context
 - ◇ Combination of visual filters, user filters, and DAX
 - ◇ Can be changed with CALCULATE()
 - ◇ Can be overridden with ALL()
 - ◇ Propagates filters to other tables

1.C.3 Common Iterator Functions

- There are many built in iterator functions, below is a small partial list of common iterators
 - ◇ SUMX()
 - ◇ MINX()
 - ◇ MAXX()
 - ◇ COUNTX()
 - ◇ COUNTAX()
 - ◇ RANKX()
 - ◇ CONCATENATEX()
 - ◇ AVERAGEX()
 - ◇ PRODUCTX()
 - ◇ CONTAINSROW()

1.C.4 Nesting Iterators

- It is a common practice to nest iterators
 - ◇ Think of this as defining multiple steps
 - ◇ Write these from the inside out
- Be aware of possible performance implications
 - ◇ Sometimes nesting iterators is the only way to solve the problem but do take some time to think of other possible strategies first
 - ◇ Example:
 - * This expression uses nested iterators to create a column ranking all products by total SalesAmount:

```
ProductSalesRank =  
RANKX(  
    ALL( Products ),  
    SUMX(  
        RELATEDTABLE( InternetSales ),  
        'InternetSales'[SalesAmount]  
    ) )
```

- * It works as written ... but a calculated column using ALL() ignores filter context *permanently*, which is rarely what a report needs
- * An AI tool asked to "rank each product by SalesAmount" might produce exactly this, without knowing that, for instance, a [Total Sales] measure already exists in your model. Instead, creating a measure for

```
RANKX( ALL( Products ), [Total Sales] )
```

would be simpler, more flexible for use in visualizations or inside other DAX calculations, and easier to maintain

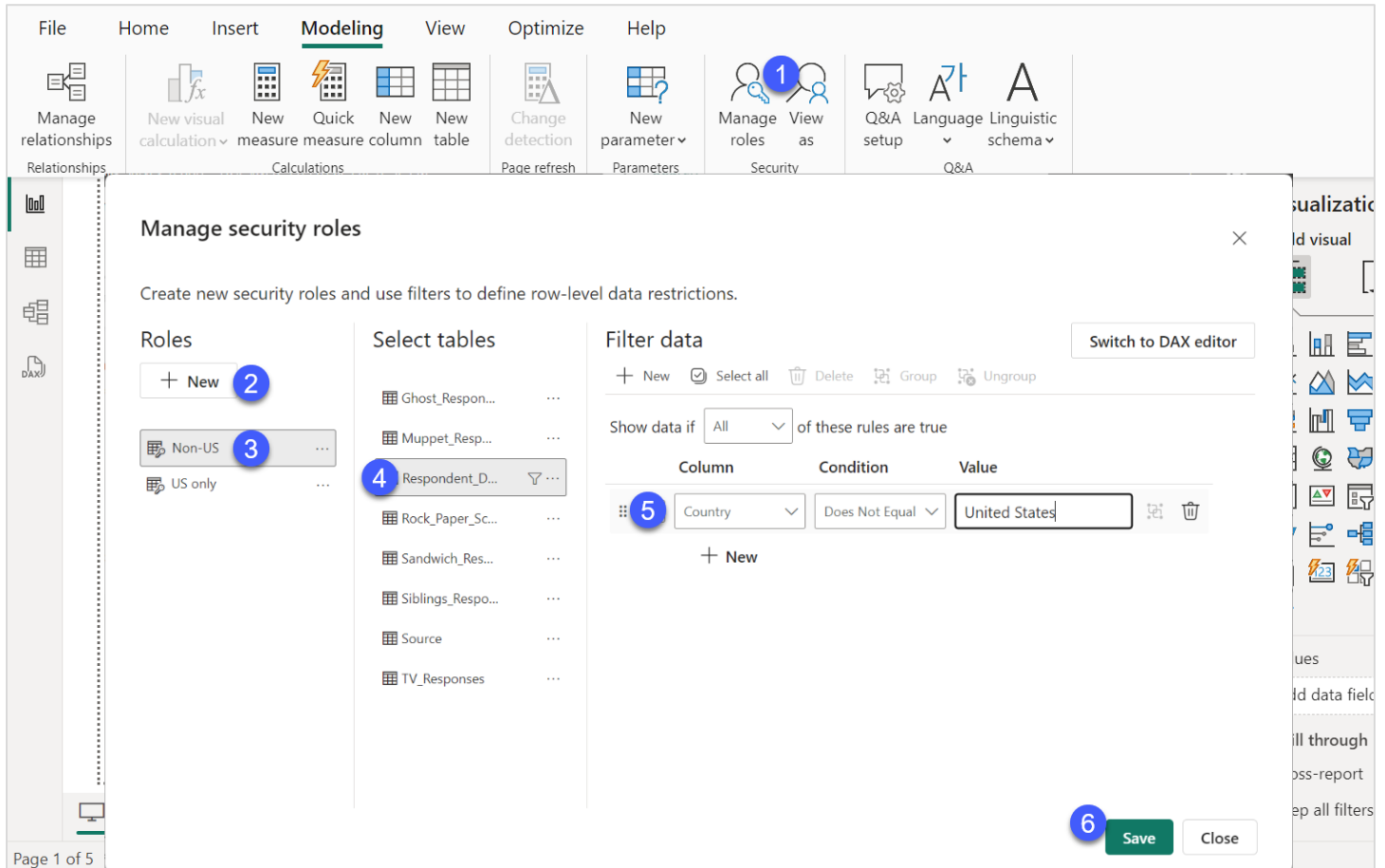
Section 1.D

Row Level Security

1.D.1 Row Level Security

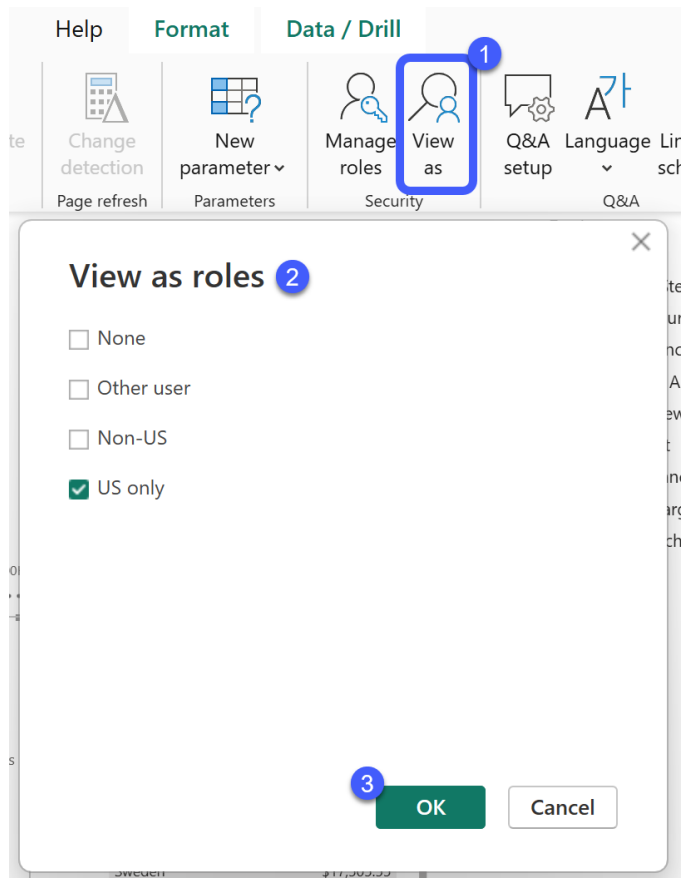
- Row Level Security can be used to restrict data access for given users, to limit a user's view to a particular subset of data
- Filters are used to restrict data access at the row level
- Filters are defined within roles
- This course speaks only to how DAX is involved in restricting a user's access to data through RLS
 - ◇ In some cases additional research should be done into workspace access rules on the underlying data source

1.D.2 Defining RLS

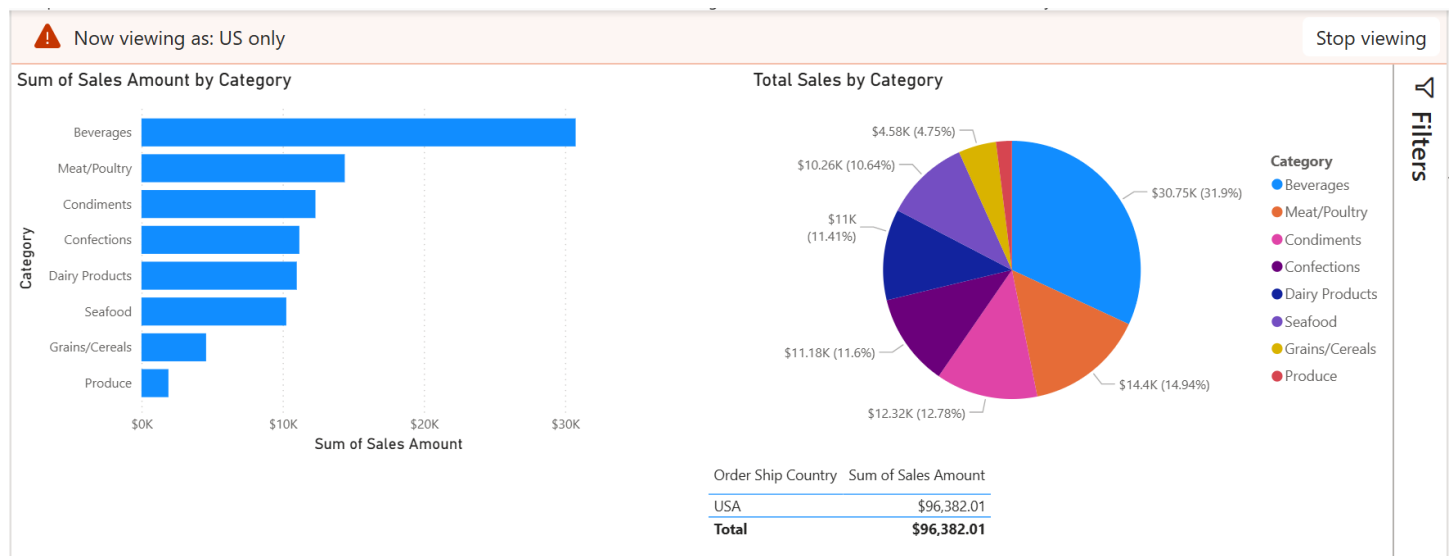


1. From the ribbon, select Manage Roles (from Modeling tab of Report view, or from Home tab of Table or Model view)
2. In the Manage Roles Dialog, Click New
3. Provide a name for the role
4. Select a table with data you want to filter on
5. Provide a DAX expression that restricts records using DAX Editor – *or* click New and use Column / Condition / Value options to build an expression
6. Click Save

1.D.3 Testing RLS



1. Select **View as** in the ribbon
2. Choose the role(s) to preview
3. Click OK, then view a report and see that data has been filtered



1.D.4 Users assigned in Power BI Service

- While roles are defined in Power BI Desktop, users or groups can only be assigned to roles in Power BI online
- This option is available from the Security settings for the semantic model (*not* at the report level)



- Users can be added one at a time by email address, or larger distribution list emails can be used to add groups of users

Row-Level Security

US ONLY (19)

Members (19)

People or groups who belong to this role

Enter email addresses

Add

Anthony Perez	×
Armando Garcia	×
aron kidane	×
Caitlan Beard	×
Craig Bowman	×
Fadi-Ilah Ashiru	×
Jacob Ridges	×

Module 2

Date and Time Intelligence

Section 2.A

Dates and Date Tables

2.A.1 Date Dimensions

- One of the most common dimensions that you will want to report on is a date
- A date field is not adequate without a date table that provides supporting attributes
- Power BI Desktop generates automatic hidden date tables for each date column
- You will frequently want to create your own, more powerful, date tables

2.A.2 Auto date/time in Power BI Desktop

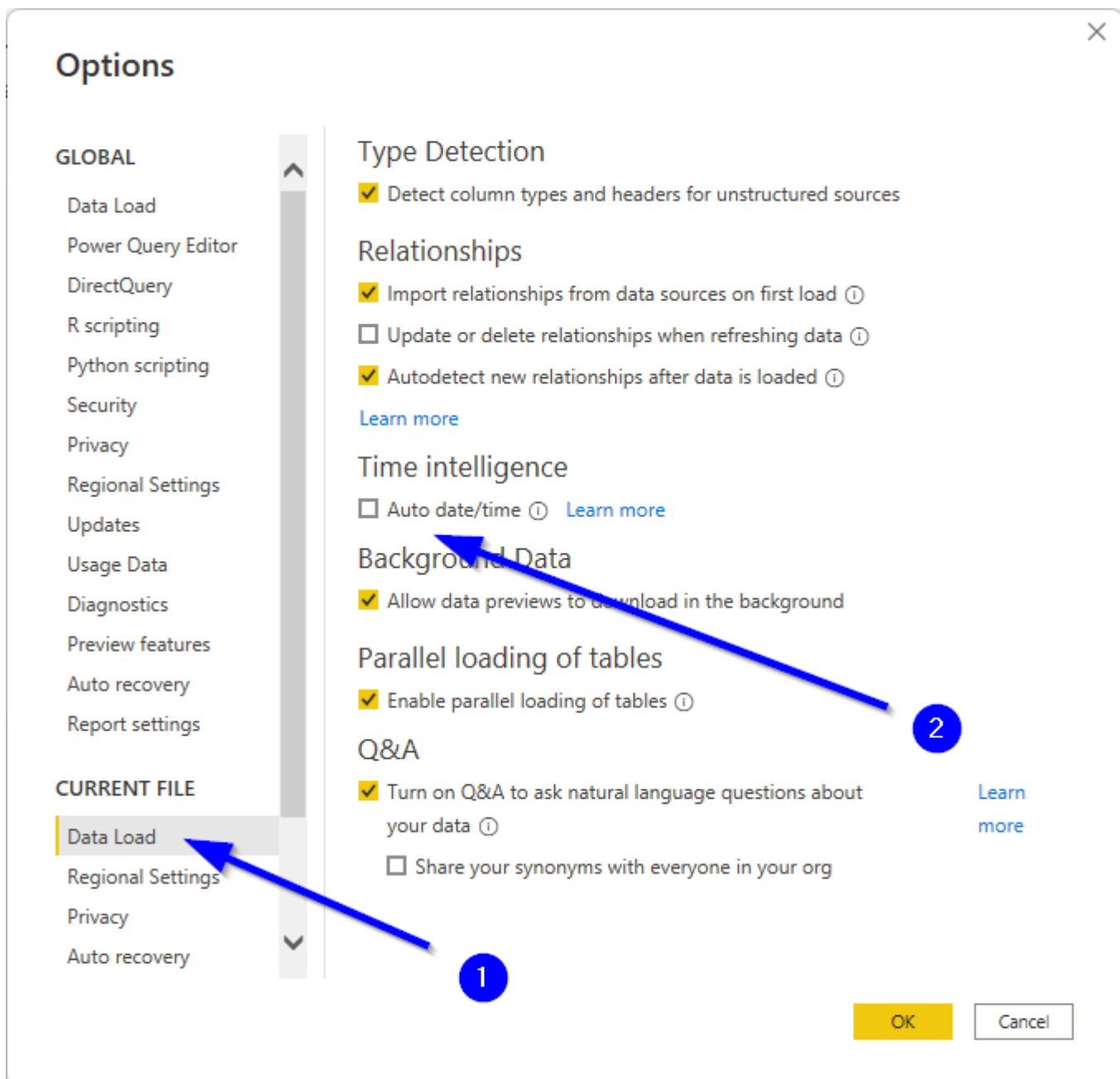
- Power BI Desktop offers an option called Auto date/time
- Auto date/time allows for convenient time intelligence reporting based on date columns loaded into a model
 - ◇ It allows report authors to filter, group, and drill down by using calendar date periods (years, quarters, months, and days)
- Auto date/time works automatically when the following conditions are met
 - ◇ The table storage mode is import
 - ◇ The column data type is date or date/time
 - ◇ The column isn't the many side of a model relationship
- Behind the scenes, the CALENDAR() function is used to create a date table with the following columns:
 - ◇ Day, MonthNo, Month, QuarterNo, Quarter, Year
 - ◇ The Model Language is followed generating month names
 - ◇ The table will contain a full calendar year encompassing all date values stored in the model date column
 - ◇ The auto date/time tables are permanently hidden, they cannot be seen
 - ◇ Date columns expand into hierarchies for selection in visualizations

2.A.3 Auto date/time limitations

- These are useful for quick single-table graphs and very basic needs
- These have limitations
 - ◇ You cannot add fields to these tables
 - ◇ They do not accommodate fiscal calendars
 - ◇ They do not include weeks
 - ◇ Two Data Tables cannot share a dimension for shared filtering capabilities
- In all but very simple scenarios, you will benefit from creating explicit date tables instead of this built-in behavior

2.A.4 Disabling Auto date/time

- If you are not going to use the Automatic Date/Time feature it is best to turn it off for your project
 - ◇ This will improve performance
- You can turn it off in the Data Load section in Options



2.A.5 Date Tables

- A **date table** is a table with one row for each day
- A date table is broken down into many columns that describe attributes of that day
 - ◇ Common examples
 - * Year
 - * Quarter
 - * Month Number
 - * Month Name
 - * Day of Year
 - * Day of Month
 - * Day of Week
 - * Day Name
 - * Week Number
 - * IsWeekday
 - * IsWeekend
- Frequently you'll see a second group of columns representing **fiscal years (FY)** versus calendar years
- Each organization may have different needs within a date table
 - ◇ An organization with fiscal years that do not align with calendar years might need a second grouping of columns
 - ◇ Some organizations may need multi-lingual descriptive columns

2.A.6 Holiday Fields

- If you do business in a single country you might add an IsHoliday field, or names of holidays
- If you do business in multiple countries, you may end up with one row for each Day/Country combination

2.A.7 Time Tables

- Time is not usually considered in a date table
- If time of day is required, this is usually a separate table
- For some organizations, a time table will have exactly 1440 records, one for each minute in a day
- For some organizations, a time table will have exactly 24 records, one for each hour in a day
- The following columns may be included
 - ◇ TimeKey
 - ◇ Hour
 - ◇ Minute
- In more complex scenarios, geography is considered, and a Daytime field is calculated to determine if there was sunlight for the period

2.A.8 Creating Date Tables

- You have a lot of different options when it comes to creating a date table
 - ◇ You could add a table to your SQL database, then import it
 - ◇ You could build data in Excel, and then import it
 - ◇ You could use a Power Query M expression to generate a date table
 - ◇ You could use a DAX expression to generate a date table
- Date tables must have an entry for every date within the range that data might exist, even if no data exists for that date
 - ◇ Some of the date functions will not function properly without this!
 - ◇ Report visualizations will skip dates where an aggregate should show 0
- Date tables should include one column of type Date with unique values
 - ◇ DateTime type is a poor choice because the time part of the field is not guaranteed to be empty
 - ◇ It is not necessary that the relationship between this table and your fact tables use the Date column
- Avoid unnecessarily creating large date ranges such as 1900-2100; this will create performance implications with some calculations
- The date table should be marked as Date in table format options

2.A.9 Making a Date Table

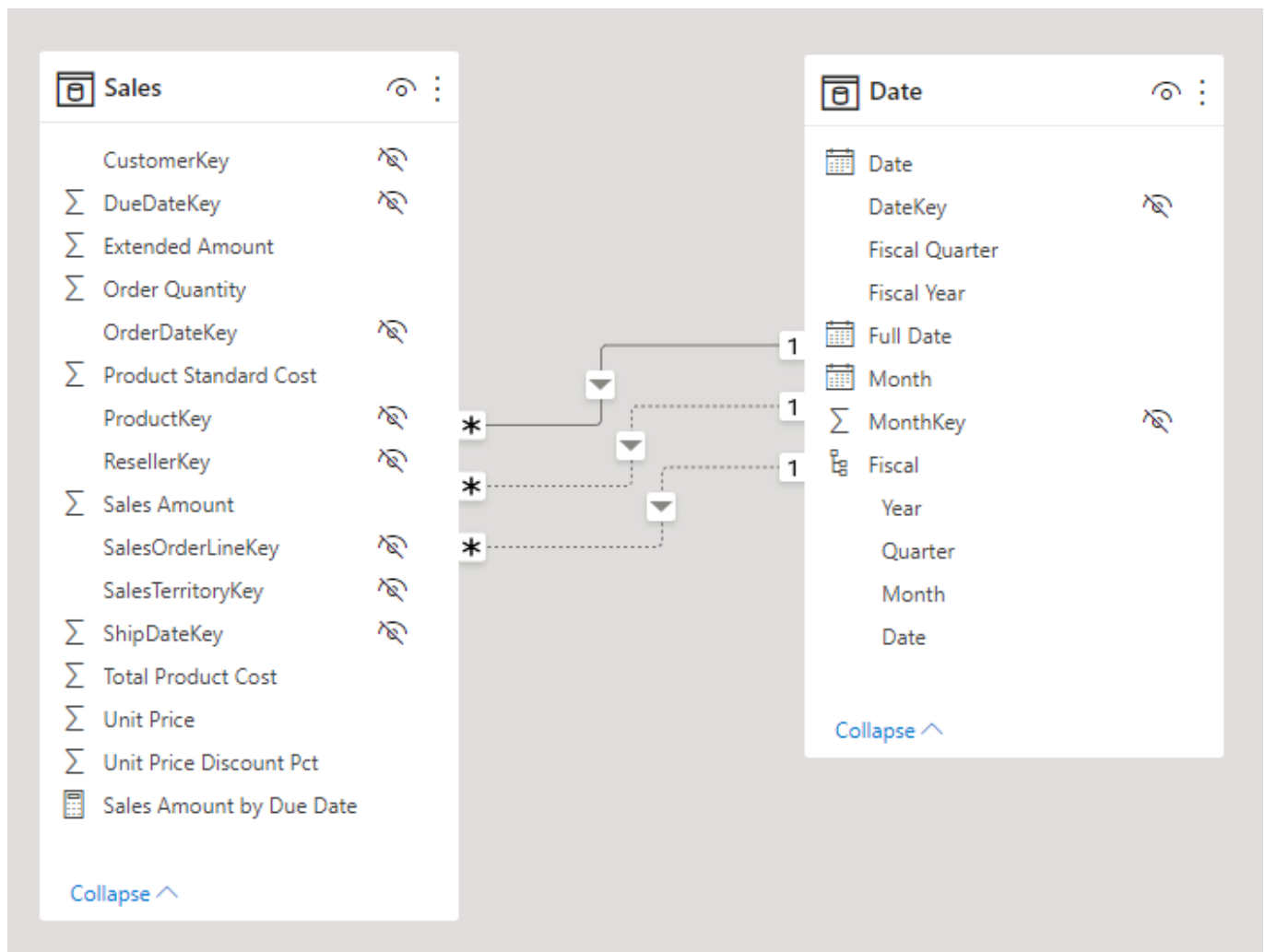
- It is important to mark your custom date tables as a date table in your data model
- This allows time intelligence functions to work properly
 - ◇ Implicitly applies ALL() to your date table
 - ◇ Only needed for non-date keys
 - * If your Key is a date, this will be inferred

The screenshot shows the Microsoft Power BI interface. The 'Table tools' ribbon is active, and the 'Mark as date table' button is highlighted. The 'Mark as a date table' dialog box is open, showing the 'Date' table selected and the 'Date' column chosen as the date column. The 'Mark as a date table' toggle is turned on. The background shows a list of dates from Saturday, June 1, 2024, to Friday, June 21, 2024.

Table: Date (180 rows)

2.A.10 Multiple Relationships and Date Tables

- It's not uncommon for there to be **multiple relationships** between a date table and a fact table
- Although Power BI will let you model these relationships, only one of them can be **active**, to avoid ambiguity in cross-filtering



2.A.11 Avoiding Ambiguous Relationships

- Ambiguity occurs when there are multiple ways to start from one table and reach another table
- DAX expressions are implicitly aware of active relationships
 - ◇ When using `RELATED()` or `RELATEDTABLE()`, there is no need to explicitly identify how the two tables relate
 - ◇ When bringing measures and attributes into visualizations, there is no need to explicitly explain how the two are related
- You can deal with multiple relationships in one of two ways
 - ◇ You can create DAX formulas that explicitly use relationships with the `USERELATIONSHIP()` function
 - ◇ You can use role-playing tables to avoid multiple relationships

2.A.12 Role Playing Tables

- **Role playing tables** are new tables for each date you'll need to represent
- To make your reports easier to read, you can change the names of the columns in each date table to represent the type of date that is being represented
- For example
 - ◇ Order Date
 - * Order Date
 - * Order Year
 - * Order Month
 - ◇ Ship Date
 - * Ship Date
 - * Ship Year
 - * Ship Month
- You can use DAX (calculated table) or Power Query (duplicate query) to duplicate a base reference table

Section 2.B

DAX Time Intelligence Functions

2.B.1 Periodized Analysis: The Hard Way

- You don't need any special time intelligence functions to perform time-based analysis
 - ◊ You already have the knowledge to write powerful expressions
 - ◊ ... but they can be complicated!
- Example
 - ◊ This illustrates how we might calculate the Year-To-Date without using any special functions:

```
Sales YTD =
  CALCULATE (
    SUM(Sales[Sales Amount]),
    FILTER(ALL(Order[OrderDate]),
      Order[OrderDate] <= MAX(Order[OrderDate])
      &&
      YEAR(Order[OrderDate]) = YEAR(MAX(Order[OrderDate]))
    )
  )
```

Pseudocode explainer:

```
Calculate:
  The sum total Sales amount from the following set:
  (Starting with every order,
  consider only the rows that meet the following conditions:
    (The specific row's order date is less than the
    greatest date found in the current filter context
    AND
    Further filter that down to only the rows with an
    order date that has the same year as the greatest
    date found in the current filter context
  )
)
```

2.B.2 Time Intelligence: The Easy Way

- Time Intelligence functions make life easier!
- There are requirements
 - ◇ You need to have a date table in your semantic model
 - ◇ The date table must include a column considered to be the date column and be marked as a date table in the modeling tools
 - * Every date **must** exist once and only once
 - * You cannot skip dates, such as skipping days with no data, or weekends
 - ◇ DAX time intelligence functions default to a standard calendar with a start date of January 1st, though some functions support fiscal years through an optional year-end date parameter (TOTALYTD and DATESYTD)
- There are three categories of Time Intelligence functions
 - ◇ Functions that return a single date
 - ◇ Functions that return a table of dates
 - ◇ Functions that evaluate expressions over a time period
- Microsoft Learn includes info on types of date-based calculations
 - ◇ Basic Date / Time Function Reference:
<https://docs.microsoft.com/en-us/dax/date-and-time-functions-dax>
 - ◇ Time Intelligence Function Reference:
<https://docs.microsoft.com/en-us/dax/time-intelligence-functions-dax>

2.B.3 Time Intelligence Examples

- Examples:

In this example we are calculating a Year-To-Date Sales Amount using the DATESYTD() function. Note this returns the same results as the Hard Way example that we provided earlier in this module:

```
Sales YTD =  
    CALCULATE(  
        [Sales Amount] , DATESYTD(Order[OrderDate])  
    )
```

A lot less work!

The following example does the same thing:

```
Sales YTD = TOTALYTD([Sales Amount], Order[OrderDate])
```

Module 3

Practical Modeling Techniques

Section 3.A

More Power Query Concepts

3.A.1 Other Transformation Functions

- People come up with all kinds of creative ways to depict data in Excel, which might be visually appealing but are not very well constructed as a data source

	A	B	C	D
1	Customer Segment	Small	Medium	Large
2	YTD 2023	\$475,633	\$562,142	\$684,521
3	H1	\$337,846	\$387,525	\$461,254
4	Q2	\$165,269	\$198,658	\$243,517
5	YTD 2022	\$470,895	\$571,236	\$678,951
6	H1	\$294,556	\$402,177	\$662,358
7	Q2	\$145,873	\$202,134	\$240,115
8				

- You may need to think creatively about how to transform pictorial spreadsheets into usable data
- Fortunately, Power Query offers many transformation tools that can help

3.A.2 Transpose

- Rotates data from rows to columns (or columns to rows) for the entire table

In Power Query, before:

	ABC 123 Column1	ABC 123 Column2	ABC 123 Column3	ABC 123 Column4
1	Customer Segment	Small	Medium	Large
2	YTD 2023	475633	562142	684521
3	H1	337846	387525	461254
4	Q2	165269	198658	243517
5	YTD 2022	470895	571236	678951
6	H1	294556	402177	662358
7	Q2	145873	202134	240115

After:

✕ ✓ *fx* = Table.Transpose(Sheet1_Sheet)

	ABC 123 Column1	ABC 123 Column2	ABC 123 Column3	ABC 123 Column4	ABC 123 Column5	ABC 123 Column6	ABC 123 Column7
1	Customer Segment	YTD 2023	H1	Q2	YTD 2022	H1	Q2
2	Small	475633	337846	165269	470895	294556	145873
3	Medium	562142	387525	198658	571236	402177	202134
4	Large	684521	461254	243517	678951	662358	240115

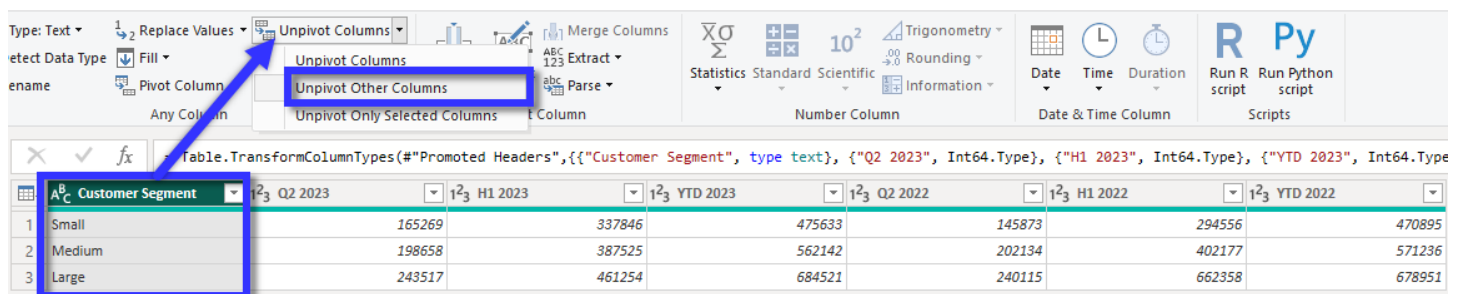
3.A.3 Pivot & Unpivot

- Pivot moves row values from selected columns into columns (rows into columns)
 - ◇ Pivot can utilize aggregation functions (sum, count, median, min/max) to cross-tabulate data
 - ◇ “No aggregation” can also be specified
- Unpivot moves values from selected columns into rows (columns into rows)
 - ◇ Reshapes data from “wide” format to a “tall” format by restacking the unpivoted columns into a single column

3.A.4 Unpivot

- Unpivot transforms selected *columns into rows*
- Former column headers become values in a new “Attribute” column, while former column values are preserved in “Value” column

Before:



	Customer Segment	Q2 2023	H1 2023	YTD 2023	Q2 2022	H1 2022	YTD 2022
1	Small	165269	337846	475633	145873	294556	470895
2	Medium	198658	387525	562142	202134	402177	571236
3	Large	243517	461254	684521	240115	662358	678951

After:

	Customer Segment	Attribute	Value
1	Small	Q2 2023	165269
2	Small	H1 2023	337846
3	Small	YTD 2023	475633
4	Small	Q2 2022	145873
5	Small	H1 2022	294556
6	Small	YTD 2022	470895
7	Medium	Q2 2023	198658
8	Medium	H1 2023	387525
9	Medium	YTD 2023	562142
10	Medium	Q2 2022	202134
11	Medium	H1 2022	402177
12	Medium	YTD 2022	571236
13	Large	Q2 2023	243517
14	Large	H1 2023	461254
15	Large	YTD 2023	684521
16	Large	Q2 2022	240115
17	Large	H1 2022	662358
18	Large	YTD 2022	678951

- (We’ll hold off an example using pivot until we walk through a few more transformation steps...)

3.A.5 Split Columns

- Easily split column data by delimiter or other attribute

Before:

	Customer Segment	Attribute	
1	Small	YTD 2023	
2	Small	H1	
3	Small	Q2	
4	Small	YTD 2022	470895
5	Small	H1_1	294556
6	Small	Q2_2	145873
7	Medium	YTD 2023	562142
8	Medium	H1	387525
9	Medium	Q2	198658
10	Medium	YTD 2022	571236
11	Medium	H1_1	402177
12	Medium	Q2_2	202134
13	Large	YTD 2023	684521
14	Large	H1	461254
15	Large	Q2	243517
16	Large	YTD 2022	678951
17	Large	H1_1	662358
18	Large	Q2_2	240115

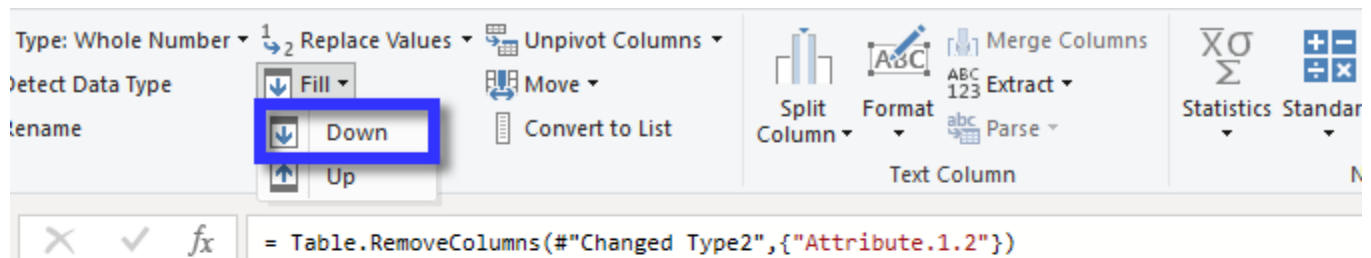
After:

	Customer Segment	Attribute.1	Attribute.2	Value
1	Small	YTD	2023	475633
2	Small	H1	null	337846
3	Small	Q2	null	165269
4	Small	YTD	2022	470895
5	Small	H1_1	null	294556
6	Small	Q2_2	null	145873
7	Medium	YTD	2023	562142
8	Medium	H1	null	387525
9	Medium	Q2	null	198658
10	Medium	YTD	2022	571236
11	Medium	H1_1	null	402177
12	Medium	Q2_2	null	202134
13	Large	YTD	2023	684521
14	Large	H1	null	461254
15	Large	Q2	null	243517
16	Large	YTD	2022	678951
17	Large	H1_1	null	662358
18	Large	Q2_2	null	240115

3.A.6 Fill

- Used to populate empty cells using the neighboring cell value in a column (up or down)

Before:



Type: Whole Number ▾ 1 2 Replace Values ▾ Unpivot Columns ▾

Detect Data Type ▾ Fill ▾

Move ▾

Convert to List

Split Column ▾

Format ▾

Merge Columns ▾

Extract ▾

Parse ▾

Text Column

Statistics ▾

Standard ▾

fx = Table.RemoveColumns(#"Changed Type2",{"Attribute.1.2"})

	A ^B _C Customer Segment ▾	A ^B _C Attribute.1.1 ▾	1 ² ₃ Attribute.2 ▾	1 ² ₃ Value ▾
1	Small	YTD	2023	475633
2	Small	H1	null	337846
3	Small	Q2	null	165269
4	Small	YTD	2022	470895
5	Small	H1	null	294556
6	Small	Q2	null	145873
7	Medium	YTD	2023	562142
8	Medium	H1	null	387525

After:

	A ^B _C Customer Segment ▾	A ^B _C Attribute.1.1 ▾	1 ² ₃ Attribute.2 ▾	1 ² ₃ Value ▾
1	Small	YTD	2023	475633
2	Small	H1	2023	337846
3	Small	Q2	2023	165269
4	Small	YTD	2022	470895
5	Small	H1	2022	294556
6	Small	Q2	2022	145873
7	Medium	YTD	2023	562142
8	Medium	H1	2023	387525

3.A.7 Pivot

- Again, pivot transforms selected *rows into columns*

◇ Before, row values in Attribute 2 are 2022 & 2023:

The screenshot shows the Power BI interface with the 'Pivot Column' button highlighted in the ribbon. Below the ribbon, a formula bar contains the formula: `= Table.FillDown("#Changed Type1",{"Attribute.2"})`. The background table has columns: Customer Segment, Attribute.1, Attribute.2, and Value. The data rows are:

	Customer Segment	Attribute.1	Attribute.2	Value
1	Small	YTD	2023	475633
2	Small	H1	2023	337846
3	Small	Q2	2023	165269
4	Small	YTD	2022	470895

The 'Pivot Column' dialog box is open, showing the following settings:

- Values Column: Value
- Advanced options:
 - Aggregate Value Function: Don't Aggregate

Buttons: OK, Cancel

◇ After, 2022 and 2023 are now columns:

	Customer Segment	Attribute.1	2023	2022
1	Large	H1	461254	662358
2	Large	Q2	243517	240115
3	Large	YTD	684521	678951
4	Medium	H1	387525	402177
5	Medium	Q2	198658	202134
6	Medium	YTD	562142	571236
7	Small	H1	337846	294556
8	Small	Q2	165269	145873
9	Small	YTD	475633	470895

Section 3.B

Combining Data Sources

3.B.1 Folder Import

- Batch import multiple data sources from a folder using Get Data > Folder
- Transform Data returns a single query containing a row for each source, with each file's contents captured in the Content column in binary format

The screenshot illustrates the steps to import data from a folder into Power BI:

- Folder Dialog:** A dialog box titled "Folder" shows the "Folder path" as "C:\Users\elizh\Documents\DemoFiles". The "OK" button is highlighted with a blue arrow.
- Folder View:** The "C:\Users\elizh\Documents\DemoFiles" folder is displayed, showing a table of files. The "Transform Data" button is highlighted with a blue arrow.
- Power BI Query:** The resulting query in the Power BI interface is shown. The query name is "DemoFiles". The formula bar shows the formula: `= Folder.Files("C:\Users\elizh\Documents\DemoFiles")`. The table has columns: Content, Name, Extension, Date accessed, Date modified, Date created, Attributes, and Folder I. The data is as follows:

	Content	Name	Extension	Date accessed	Date modified	Date created	Attributes	Folder I
1	Binary	folder_import_2_demo.xlsx	.xlsx	4/14/2023 4:34:19 PM	4/14/2023 4:33:38 PM	4/14/2023 4:34:19 PM	Record	C:\Users\elizh\Doc
2	Binary	folder_import_demo.xlsx	.xlsx	4/14/2023 4:34:19 PM	4/14/2023 3:05:21 PM	4/14/2023 4:34:19 PM	Record	C:\Users\elizh\Doc
3	Binary	Pivot-Unpivot Sample.pbix	.pbix	4/14/2023 4:34:19 PM	4/13/2023 8:00:01 PM	4/14/2023 4:34:19 PM	Record	C:\Users\elizh\Doc
4	Binary	pivot_demo.xlsx	.xlsx	4/14/2023 4:34:19 PM	4/14/2023 3:03:27 PM	4/14/2023 4:34:19 PM	Record	C:\Users\elizh\Doc

3.B.2 Combine

- Merges multiple tables into a single table by combining based on one or more common columns
- Results in a single table with all the columns from all the tables

Original Excel:

	A	B	C	D	E	F	G
1	15 Top Performing Sales Reps						
2	First Name	Last Name	Segment Q1 Sales	Location	State	Territory	Segment
3	Maya	Ivanov	34%	Pittsburgh	PA	Northeast	Large
4	Sofia	Smith					
5	Taylor	Kim					
6	Nina	Nascimento					
7	Kai	Yamamoto					

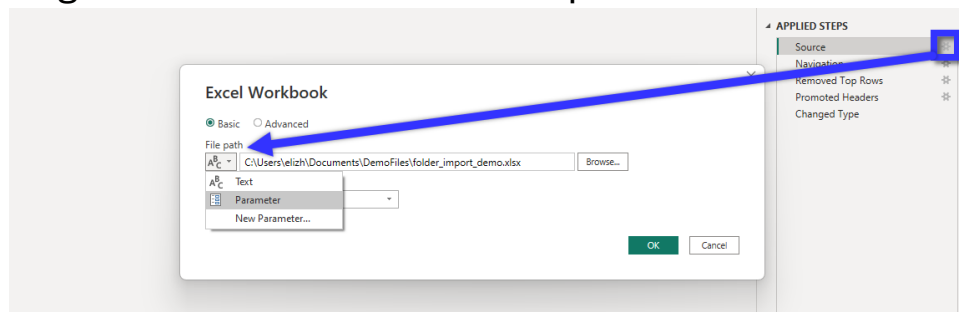
	A	B	C	D	E	F	G
1	15 Top Performing Sales Reps						
2	First Name	Last Name	Segment Q2 Sales	Location	State	Territory	Segment
3	Sofia	Smith	33%	Boise	ID	Northwest	Medium
4	Liam	Patel	33%	Eugene	OR	Northwest	Large
5	Maya	Ivanov	32%	Pittsburgh	PA	Northeast	Large
6	Kai	Yamamoto	31%	Bismarck	ND	North Central	Small
7	Taylor	Kim	30%	Rochester	NY	Northeast	Small

In Power Query, after using Combine:

Source.Name	A1: 15 Top Performing Sales Reps	A1: Column2	A1: Column3	A1: Column4	A1: Column5	A1: Column6	A1: Column7
1	First Name	Last Name	Segment Q2 Sales	Location	State	Territory	Segment
2	Sofia	Smith	0.3278	Boise	ID	Northwest	Medium
3	Liam	Patel	0.326	Eugene	OR	Northwest	Large
4	Maya	Ivanov	0.32	Pittsburgh	PA	Northeast	Large
5	Kai	Yamamoto	0.31	Bismarck	ND	North Central	Small
6	Taylor	Kim	0.3042	Rochester	NY	Northeast	Small
7	Isabella	Ng	0.2774	Culver City	CA	West	Large
8	Cameron	Lopez	0.27	Phoenix	AZ	West	Large
9	Avery	Andersson	0.2523	Tallahassee	FL	South	Medium
10	Nina	Nascimento	0.251	Las Vegas	NV	West	Large
11	Alex	Gonzalez	0.24	Atlanta	GA	South	Large
12	Ayaan	Dubois	0.2324	Wilmington	DE	Northeast	Medium
13	Jackie	Fisher	0.23	Sioux Falls	SD	North Central	Small
14	Luka	Kowalski	0.2167	Nashville	TN	South	Small
15	Dylan	Gardner	0.21	Tacoma	WA	Northwest	Large
16	Jennifer	Beanman	0.19	Iowa City	IA	North Central	Small
17	First Name	Last Name	Segment Q2 Sales	Location	State	Territory	Segment
18	Maya	Ivanov	0.3398	Pittsburgh	PA	Northeast	Large
19	Sofia	Smith	0.3278	Boise	ID	Northwest	Medium
20	Taylor	Kim	0.3042	Rochester	NY	Northeast	Small
21	Nina	Nascimento	0.2945	Las Vegas	NV	West	Large

3.B.3 Custom Functions & Parameters

- Custom Functions are reusable pieces of code that can be called on by queries
- Parameters are reusable values for use in queries
- Helpful for reducing repetitive steps and duplicated code, and for simplifying the maintenance and reuse of your queries
- Example:
 - ◇ In the scenario of multiple matching spreadsheets, first one is transformed to get the data into usable format
 - ◇ These steps can be repurposed as a Custom Function, replacing the original source location with a parameter



or



Module 4

Advanced Analytics with Power BI

Section 4.A

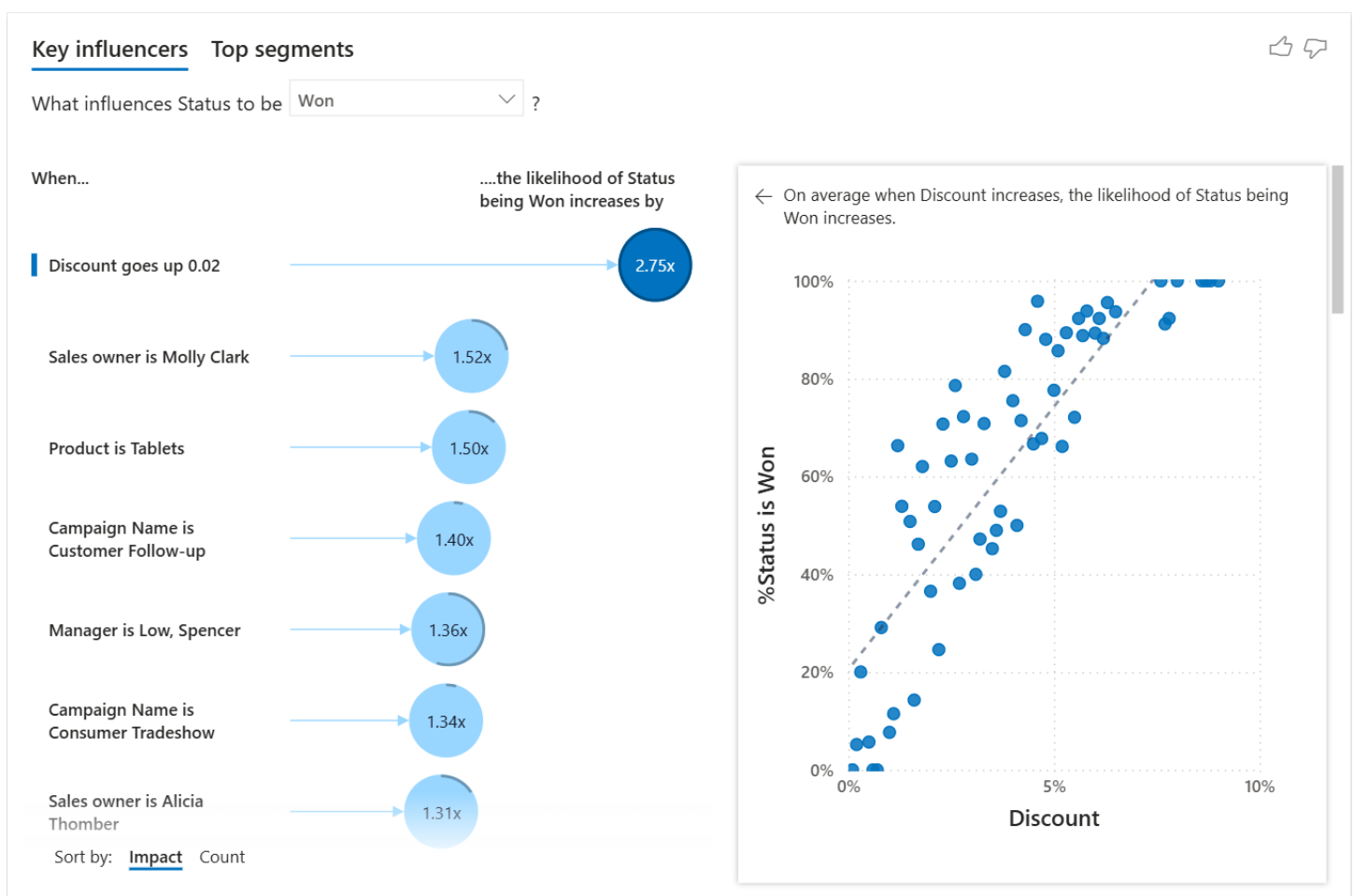
Advanced Analytics Features

4.A.1 Overview of Advanced Analytics

- Power BI offers a wide and ever-increasing array of analytics features utilizing machine learning and AI capabilities
- AI-driven features automatically analyze data and provide meaningful insights and visualizations to uncover hidden patterns, trends, and anomalies in your data without requiring advanced statistical knowledge

4.A.2 Key influencers visualization

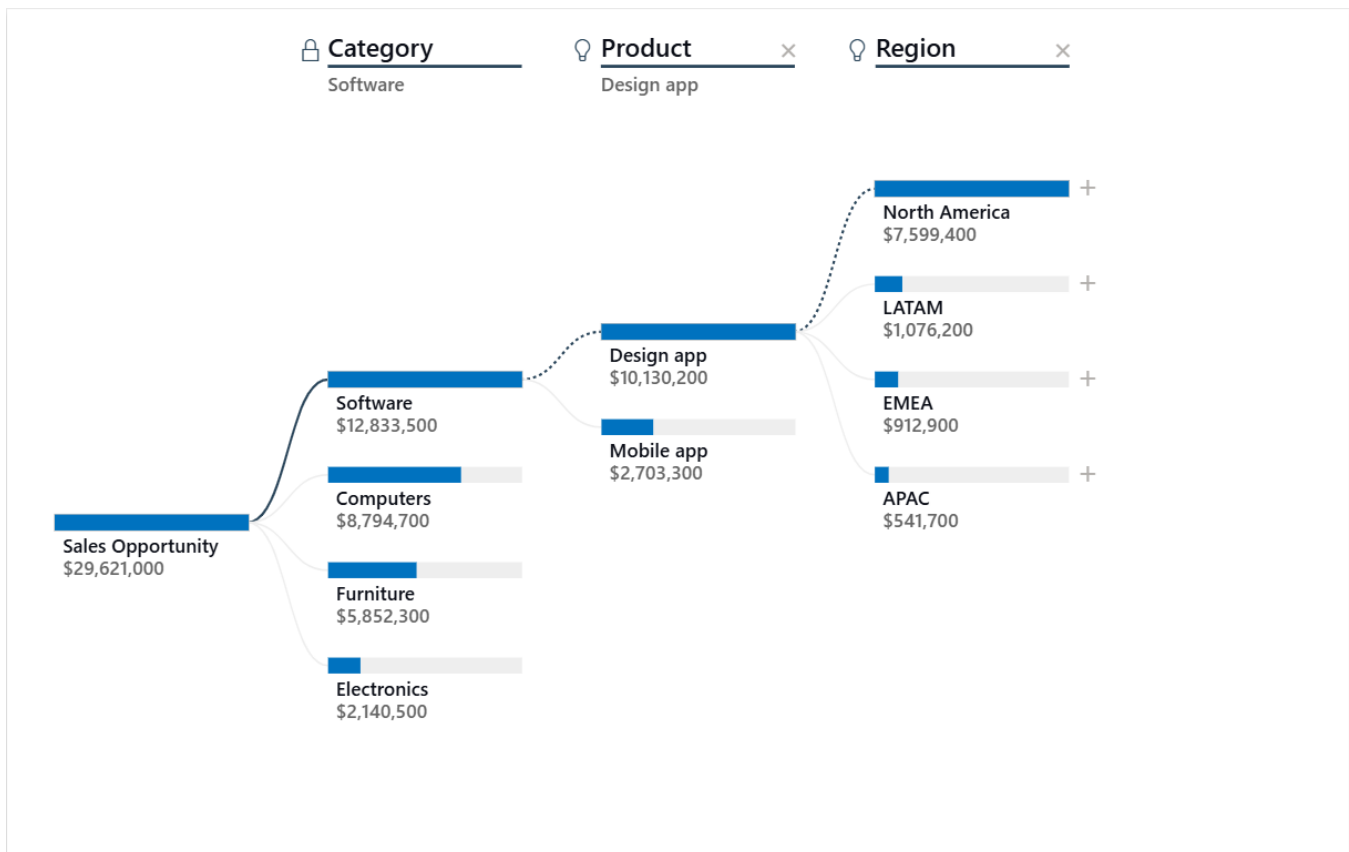
- Power BI's key influencers visualization analyzes your data, ranks the factors that matter, and displays them as key influencers
- Can be a useful tool to identify factors that affect a given metric, and to compare the relative importance of those identified factors



Example from the Artificial Intelligence Sample report

4.A.3 Decomposition tree visualization

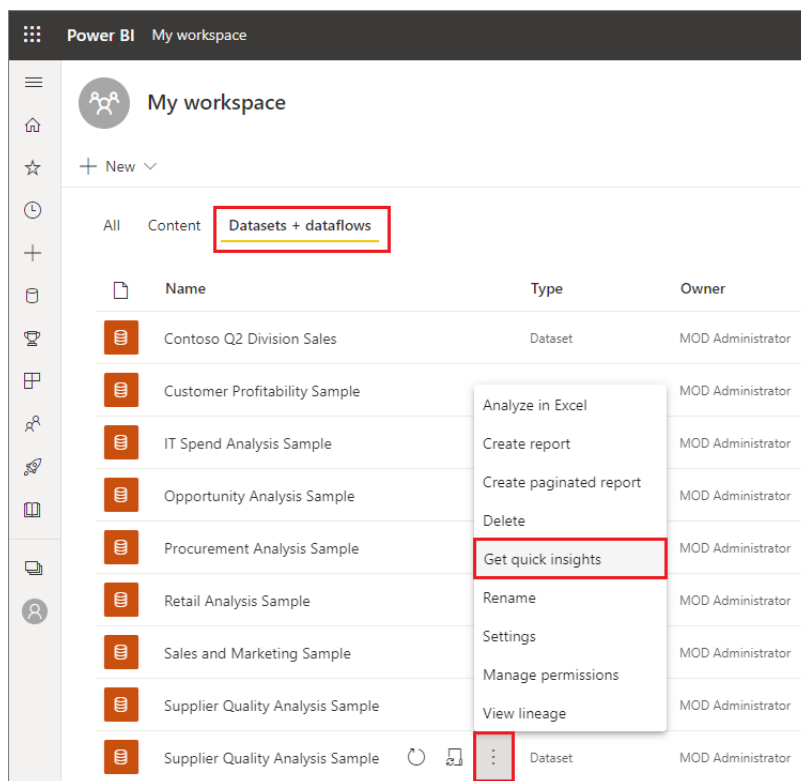
- The decomposition tree enables visualization of data across multiple dimensions in (as the name suggests) a tree format
- Once dimensions are added, the user can drill down through the dimensions in any order
 - ◇ This makes the decomposition tree a particularly useful tool for data exploration and ad hoc analysis, and also for root-cause analysis



Example from the Artificial Intelligence Sample report

4.A.4 Quick Insights (Power BI Service)

- The Quick Insights feature utilizes machine learning to analyze your data and generate insights
- Can help uncover hidden patterns and trends in data that might otherwise have been overlooked
- Also helps you identify outliers and anomalies in your data that might be indicative of errors or other issues
- Provides a convenient way to kickstart exploration or identify areas where further analysis is needed



4.A.5 Types of Quick Insights

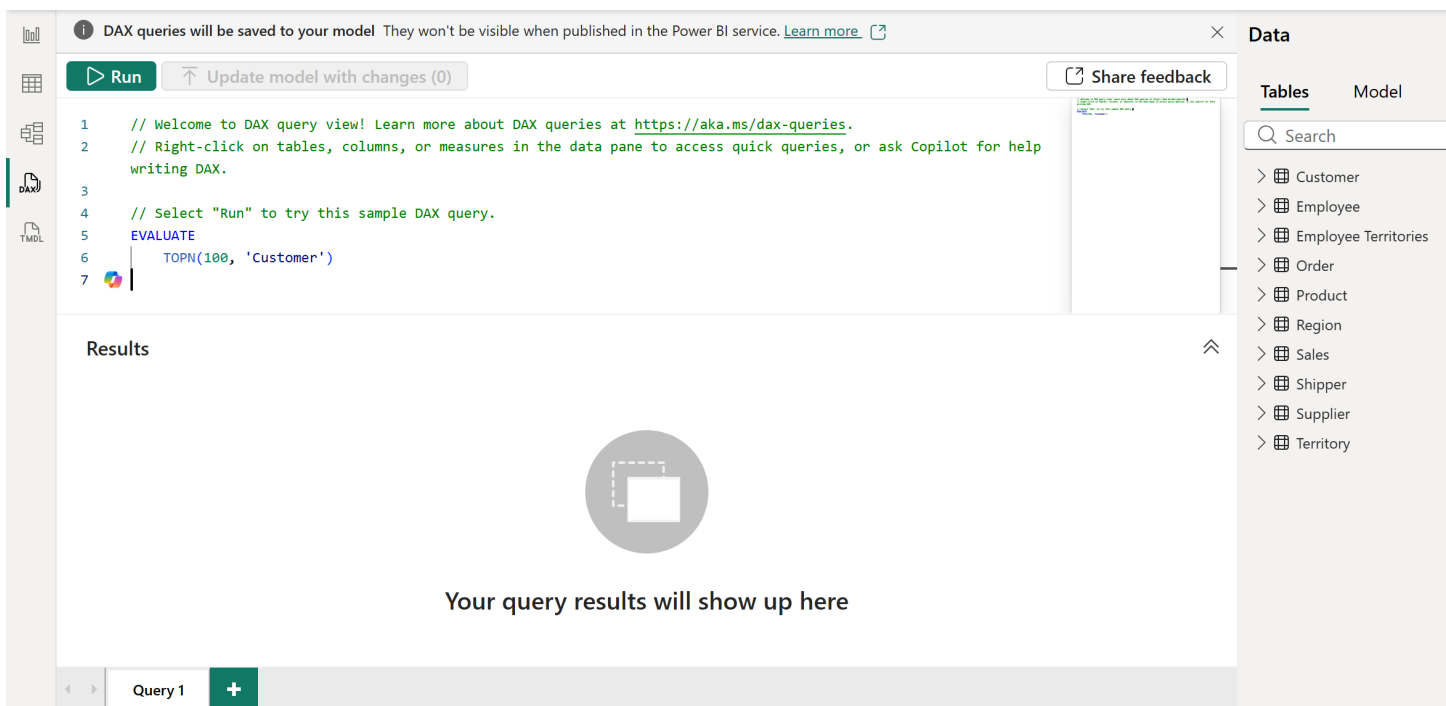
- Types of insights include:
 - ◇ Category outliers (top/bottom)
 - ◇ Outliers (based on clustering)
 - ◇ Correlation
 - ◇ Low variance from the mean
 - ◇ Majority (Major factors)
 - ◇ Steady share
 - ◇ Change points in a time series
 - ◇ Overall trends in time series
 - ◇ Seasonality in time series
 - ◇ Time series outliers

4.A.6 Using Copilot with Power BI

- Copilot can be used within Power BI for advanced AI assistance
 - ◇ Use of Copilot requires a paid Microsoft Fabric subscription capacity
 - ◇ The standard Power BI Pro license and Premium Per User license *don't* include Copilot
- Copilot relies on semantic model metadata to interpret natural language queries
 - ◇ Metadata includes field descriptions, measure definitions, naming conventions, and relationships
 - ◇ This means that if you plan to use Copilot, investing time in writing good descriptions for tables, columns, and measures can pay off significantly
 - ◇ If your model includes messy table and column names with no supporting descriptions in the metadata, you will find it much harder to get useful, meaningful results with Copilot
- Starting in 2027, the old Q&A visualization will no longer be available, meaning that Copilot will be the only natural language option for querying data in Power BI going forward
 - ◇ Organizations that currently use the old Q&A visualization in reports or dashboards may need to spend some time updating their model metadata in order to switch over to Copilot going forward

4.A.7 Other data exploration and analysis tools

- The **DAX Query View** is a newer feature that allows you to quickly query your data with DAX
 - ◇ This is a little similar to querying data with SQL, but using DAX syntax
 - ◇ Queries can be saved to the model but will not be visible to end users of the published report, meaning saved queries are not visible in Power BI online



- Similarly, Microsoft recently added a **TMDL view** to Power BI which can be used to script, modify, and apply changes to semantic model objects
 - ◇ TMDL (Tabular Model Definition Language) is primarily a developer-facing tool set which can be valuable if you're working with source control (Git), collaborating on models with a team, or want to make bulk edits to your semantic model outside of the Power BI Desktop GUI

Module 5

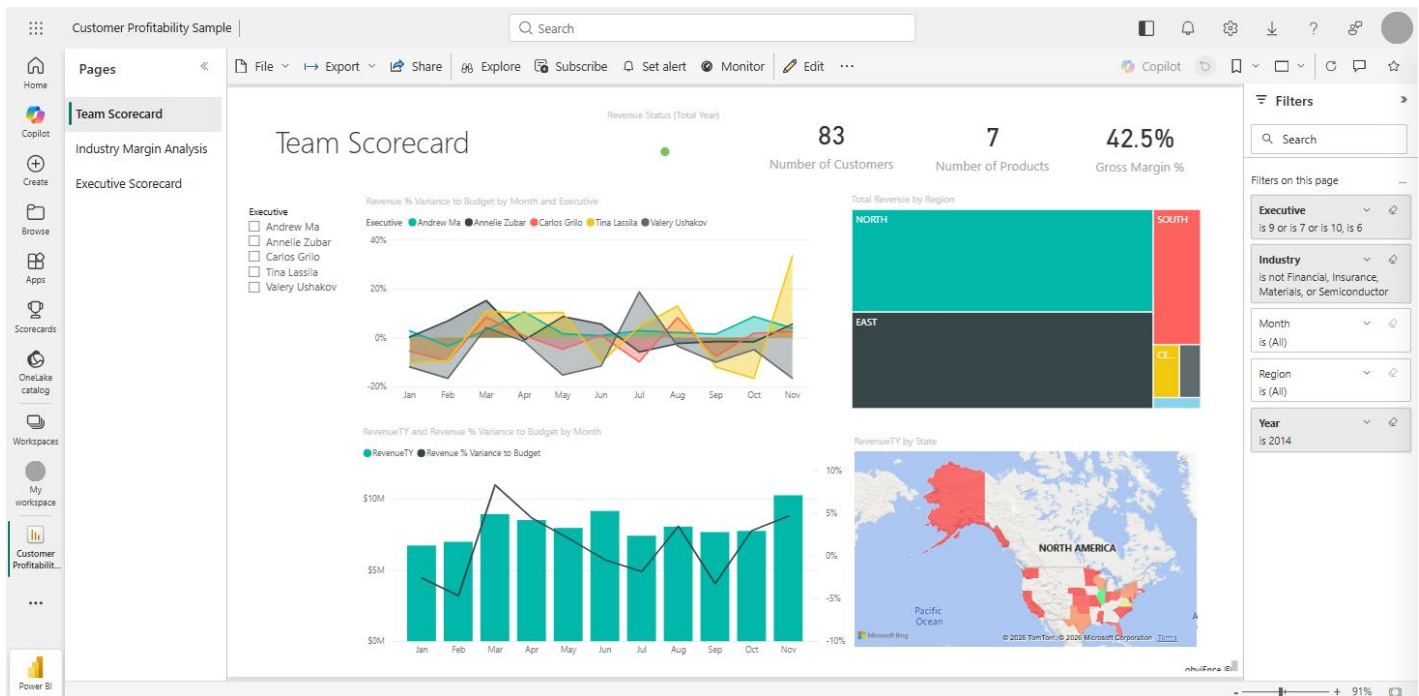
Sharing Power BI Reports

Section 5.A

Power BI Service

5.A.1 Power BI Service

- Give others view or edit access to your report via “Publish” in Power BI Desktop
- Ensures that everyone is looking at the same version of the report based on current data, reducing the risk of errors and inconsistencies
- Allows secure sharing outside your organization, such as to clients or partners, while keeping control of data
- Full interactivity and functionality are available to end users of the report

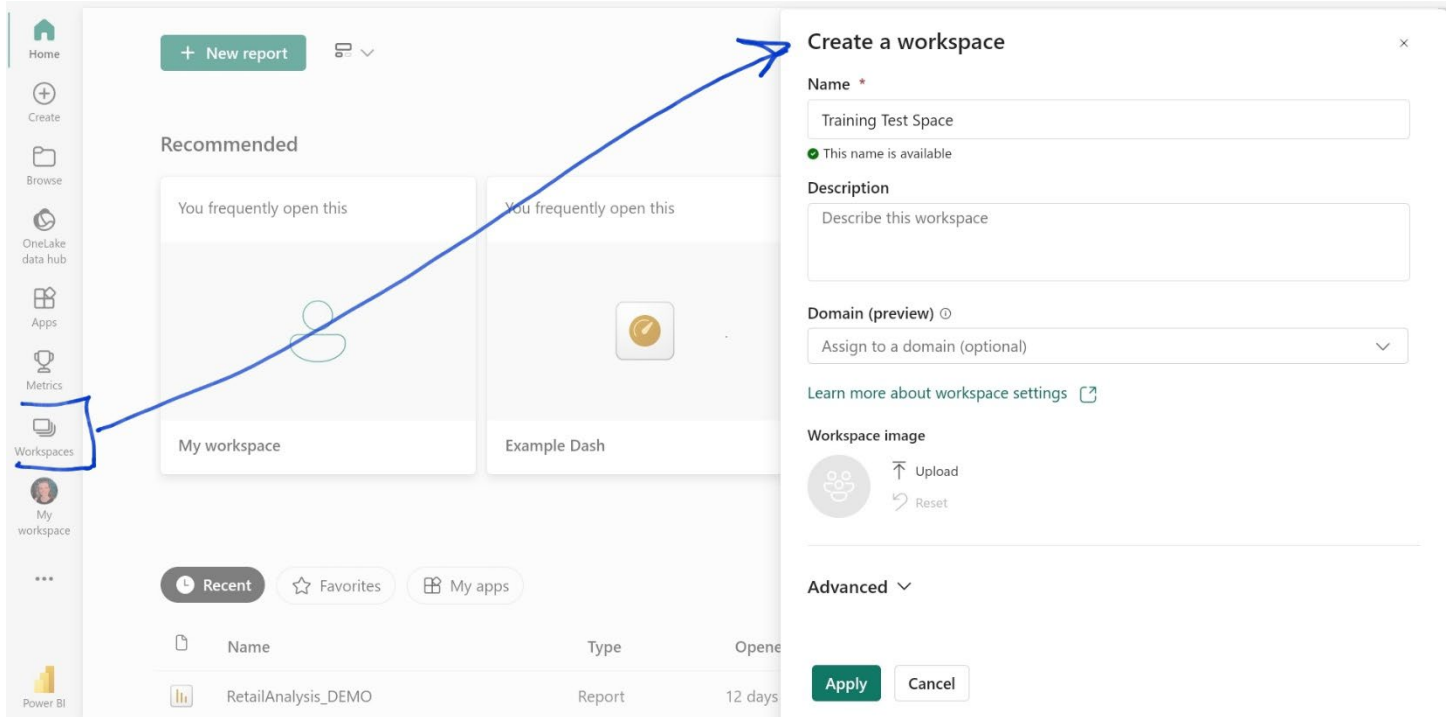


5.A.2 Collaborate Using Workspaces

- When you publish to Power BI service, you will do so within a **workspace**
 - ◇ Workspaces allow teams to collaborate with colleagues on specific content
 - ◇ Shared workspaces are created to hold collections of data, reports, and dashboards
 - ◇ A shared workspace must be created under a Pro or Premium subscriber capacity
- Everyone using the Power BI service also has a default workspace called “My Workspace”
 - ◇ This is a personal sandbox where you can create and test out content, but not share or collaborate with others

5.A.3 Publishing to a Workspace

- Before initiating the Publish workflow in Power BI desktop, either you or another user in your team will need to create a shared workspace for the project



- This workspace will then be available as a destination when publishing from Power BI desktop

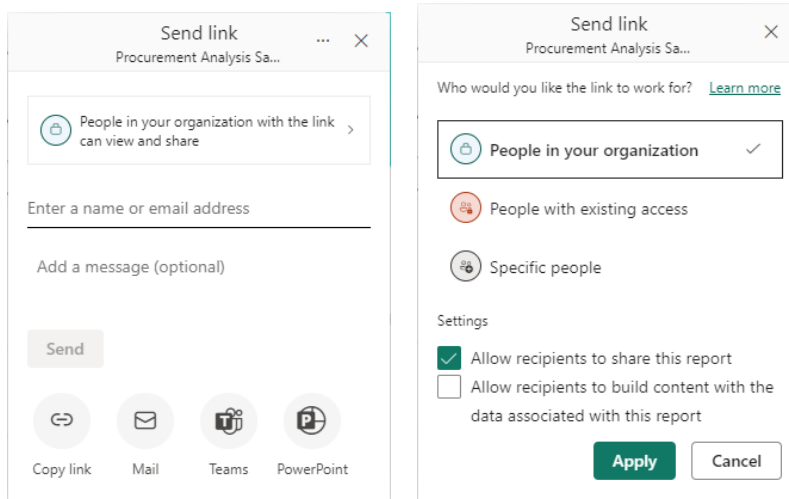
5.A.4 Tailoring Report for Mobile Sharing

- Two ways to format your report for mobile display
 - ◇ In Power BI Desktop: from **View** tab > **Mobile layout**
 - ◇ In Power BI Service: using **Edit** > **Mobile layout**
- It provides users with report access a better experience when viewing the report in the Power BI mobile app.

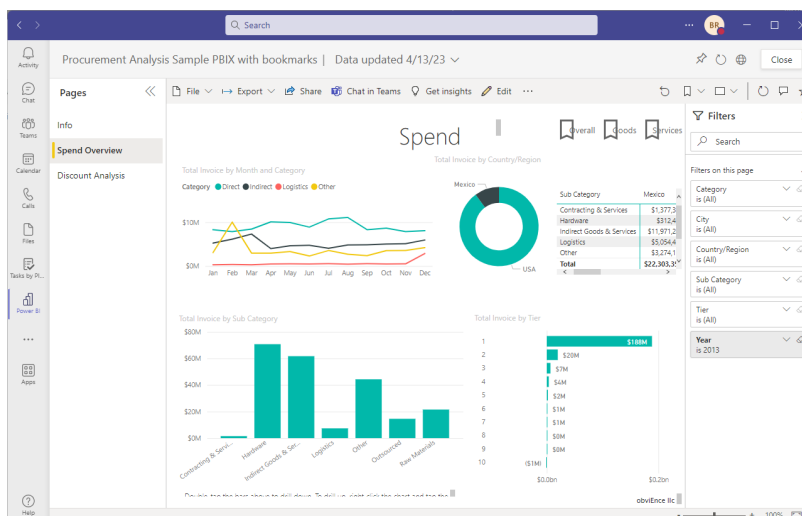


5.A.5 Sharing from within Power BI Service

- Share options are similar to other Microsoft products
 - ◇ Also includes ability to allow/restrict access to the underlying data for other analysis.

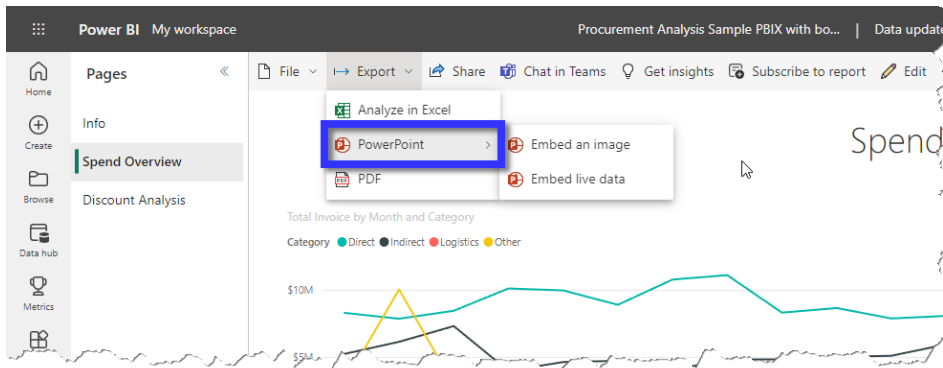


- Report can also be shared to Teams or PowerPoint
 - ◇ If shared to a Teams channel, report can be pinned as a tab



5.A.6 Export to PowerPoint

- Send report to PowerPoint via **Export** or **Share** tabs.



- Export to a static file or with live data embedded

◇ Embed an image

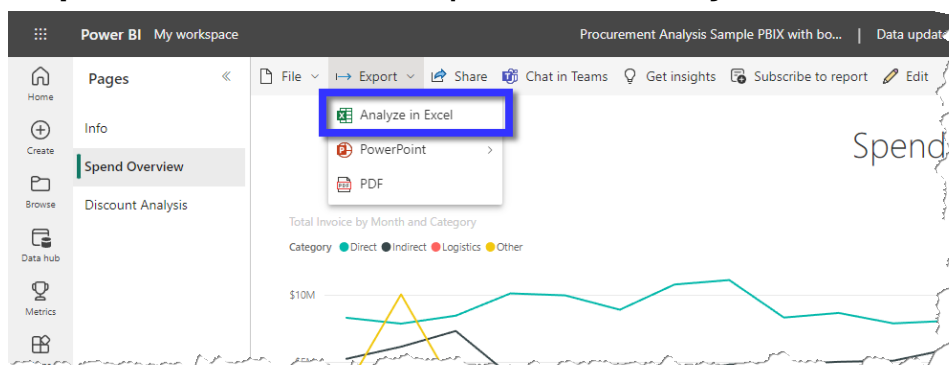
- * Allows export of full report or single page
- * Includes auto-generated cover page with link to online report
- * Reformats canvas to 4:3, which may squash or conceal elements
- * Alt text for visualizations / images appears in **Notes** section

◇ Embed live data > Open in PowerPoint

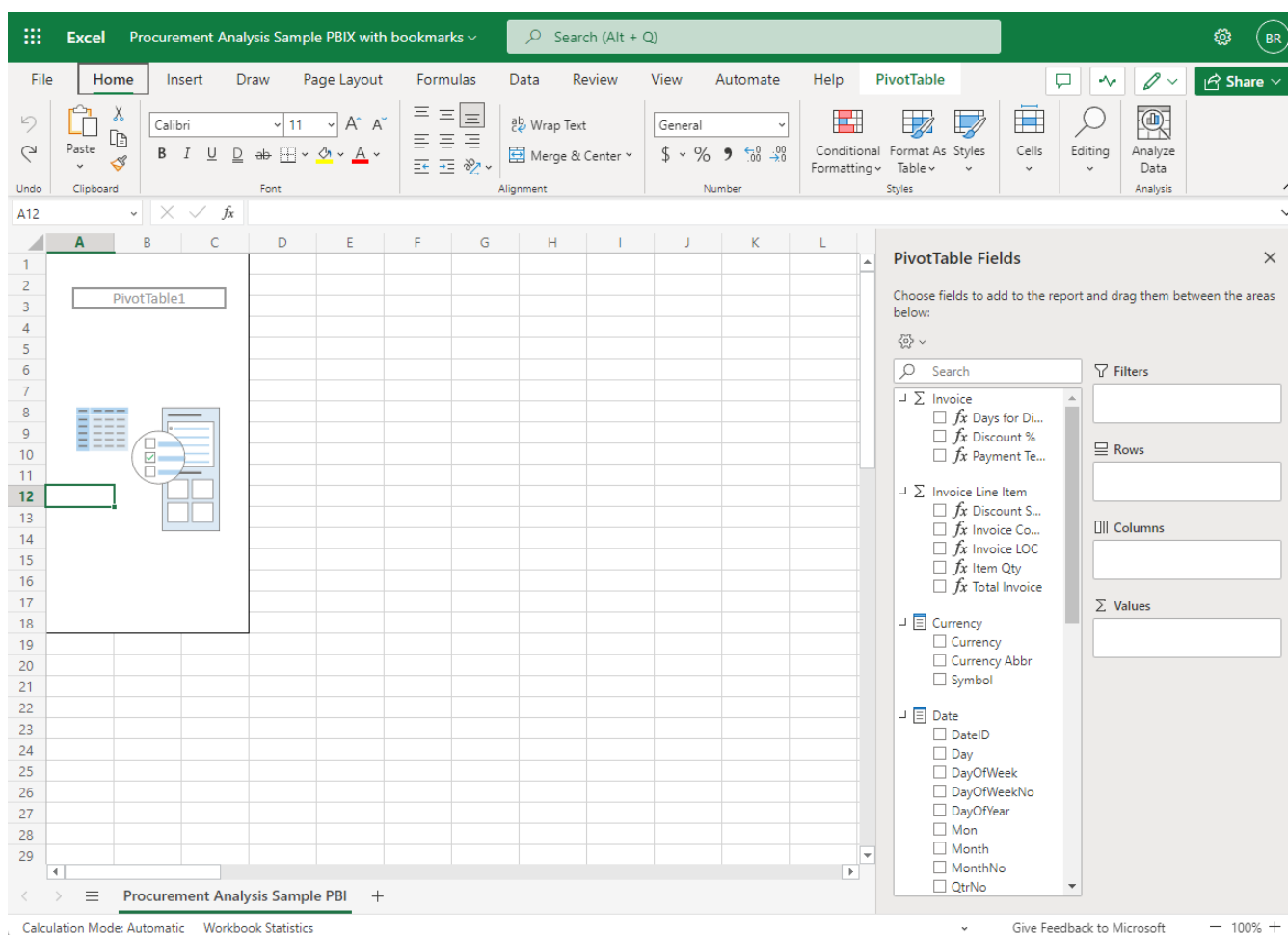
- * Requires add-in for PowerPoint (auto installed)
- * Exports single page including filter capabilities

5.A.7 Analyze in Excel

- Export tab includes option to Analyze in Excel



- Gives users ability to do their own pivot table analysis

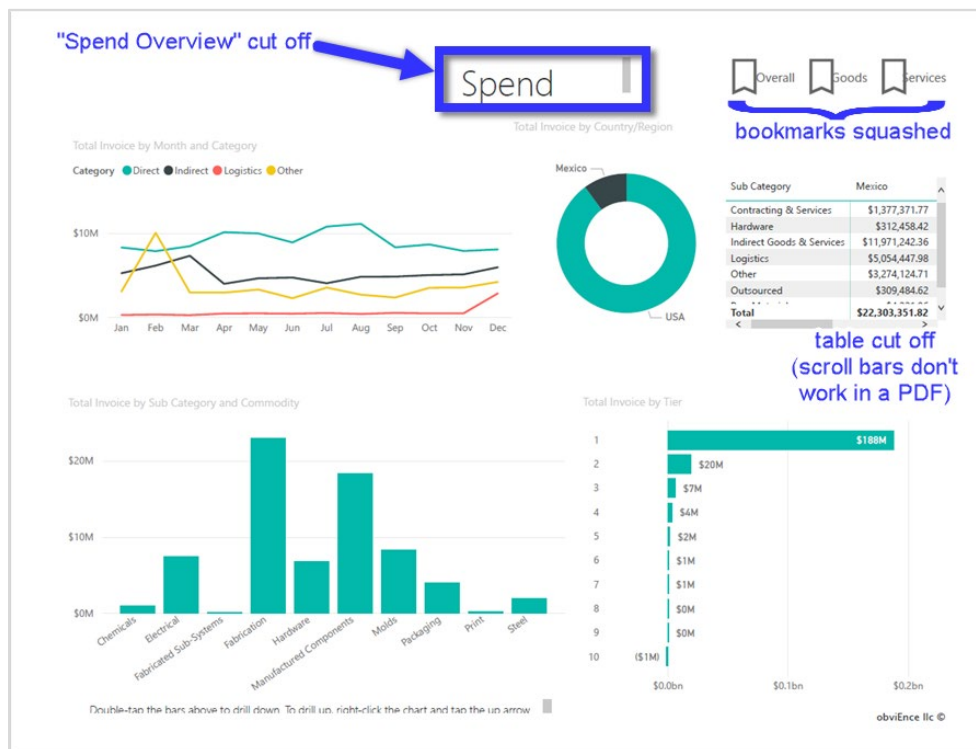


Section 5.B

Other Report Export Options

5.B.1 Exporting to PDF

- Available directly from Power BI Desktop to create a flat export of a report
- ◇ **File > Export > Export to PDF**
- Exports report pages as-is based on the current report state, including applied filters
- May reformat canvas to 4:3 (letter size), which may squash or conceal elements like text boxes, tables, or navigation buttons
- ◇ If you know the report recipient will be printing it, plan for this in your page formatting from the outset!



5.B.2 Exporting a Template

- An existing report can also be used to create a template
 - ◇ **File > Export > Power BI template**
- Contains all the pages, visuals, and formatting along with data model items and queries that are in the report on which the template is based
- Useful for recurring business reports or dashboards, where the same design and formatting is needed on a regular basis
- Starting from template ensures new copy of report is created, avoiding accidental overwrite of prior version

Section 5.C

Other Power BI Service Features

5.C.1 Beyond Report Sharing

- While we previously covered the basics of sharing reports from Power BI Desktop, there's a whole lot more to Power BI service
 - ◇ Robust tools for collaboration and data sharing
 - ◇ Online creation of dashboards and reports
 - ◇ Online tools for modeling (dataflows) and analysis
 - ◇ AI-assisted data insights (Quick Insights, Copilot)
 - ◇ Features supporting data security and compliance

5.C.2 Power BI Service Basics

- Main landing page includes quick links to access the many compartments of the service
- To create a Power BI report online:
 - ◇ **Create** and **New report** both kick off report creation workflow, using new data or an existing data set
 - ◇ Using existing data, option to start from a blank report canvas or use **Auto-create report**
 - ◇ In **Edit** mode, visualizations and filters can be added and edited much the same as in Power BI Desktop
- However, there's also much more to Power BI service...

5.C.3 Dataflows

- Essentially, Dataflows provide the capabilities of Power Query within Power BI service
- Create reusable transformation logic that can be shared by many datasets and reports inside Power BI
- Limit access to underlying data sources and curate a single source of truth for use across your organization
- Can work with other services and products in the Power Platform
- Persist data in your own Azure Data Lake Gen 2 storage, enabling you to expose it to other Azure services outside Power BI
- Gives administrators finer control over data refresh operations

5.C.4 Dashboards

- A dashboard is a powerful single-page display that can be created and shared in Power BI service
 - ◇ Made up of tiles, which can be sourced from other existing reports
 - ◇ Allows cross-view of multiple datasets from a single dashboard
 - * Tile navigation allows quick linking from visual back to source report, for further exploration of that dataset
 - * Live pages (pinned report pages) also support interactions like use of slicers and filters
- Dashboards support alerting for subscribed end-users
- To view dashboards, user must have a Pro or Premium Per User (PPU) license, or the content must be shared from a workspace in Premium capacity

5.C.5 Workspaces

- Shared workspaces enable collaboration and easy sharing of reports, dashboards, datasets, etc.
- Specify contact list of users to add to a workspace, or add workspace to a OneDrive location
 - ◇ Power BI Premium users may choose to make the workspace a part of a dedicated capacity as opposed to sharing computational resources with others
- Individuals or groups added to a workspace can be assigned role of Admin, Member, Contributor, or Viewer
- Content can be published to your workspace as an “app” that is comprised of a collection of reports, dashboards, and datasets
 - ◇ App may be made accessible to your entire organization, or specific users and groups, depending on settings configured by your org’s Power BI Administrator

5.C.6 Admin Role for Workspaces

- Users with a role of Admin can manage other users' access to a workspace
 - ◇ View, add, or remove members
 - ◇ Change role level of workspace users
- Workspace admins also have access to other settings
 - ◇ Includes options to add/remove a dedicated workspace OneDrive, manage integration with Azure or Git, and manage app and data settings for the workspace

Workspace settings [X]

About (selected)

Premium

Azure connections

System storage

Git integration

Other

Power BI (expanded)

General (selected)

Data connections

Embed codes

Workspace image

Upload

Reset

Name

Project Arrow

Description

Describe this workspace (Optional)

Domain (preview)

Assign to a domain (optional)

Contact list

Bess Hambleton X AC Ana Castillo X

Workspace OneDrive

(Optional)

5.C.7 Apps in Power BI Service

- Apps may also refer to pre-built reports and dashboards which may be connected to your own data
- An organization may also create their own library of internal apps, in addition to those externally available from AppSource
 - ◇ Apps created by third-party developers can also be released on AppSource for others' use
- Users can install these templates using the **Get apps** button

5.C.8 Deployment pipelines

- Allows a user to create a pipeline to manage workspace content through deployment
- The workplace associated with the pipeline can be assigned to a deployment stage (development, test, or production)
- Allows previewing and management of workspace content up to the point of readiness for release to end users
- For this course, you do not have to worry about creating or managing deployment pipelines -- just know that they exist, in case you find yourself working in a team that uses this feature

5.C.9 Data Hub

- OneLake Data Hub allows users across an organization to easily share, use, and manage data
- Provides quick visibility into other reports created using shared data sets
- Share settings can be customized to open up or lock down access to data
 - ◇ Tables and/or selected columns from a data set can be quickly exported where sharing is active
- Links to data lineage view within the workspace where that data is housed

Module 6

Data Governance

Section 6.A

Data Governance Basics

6.A.1 Introducing Data Governance

- Data Governance is a broad and complex topic
 - ◇ In this course, we will discuss it as a high-level overview

- Defined by the Data Governance Institute as:

“a system of decision rights and accountabilities for information-related processes, executed according to agreed-upon models which describe who can take what actions with information, and when, under what circumstances, using what methods”

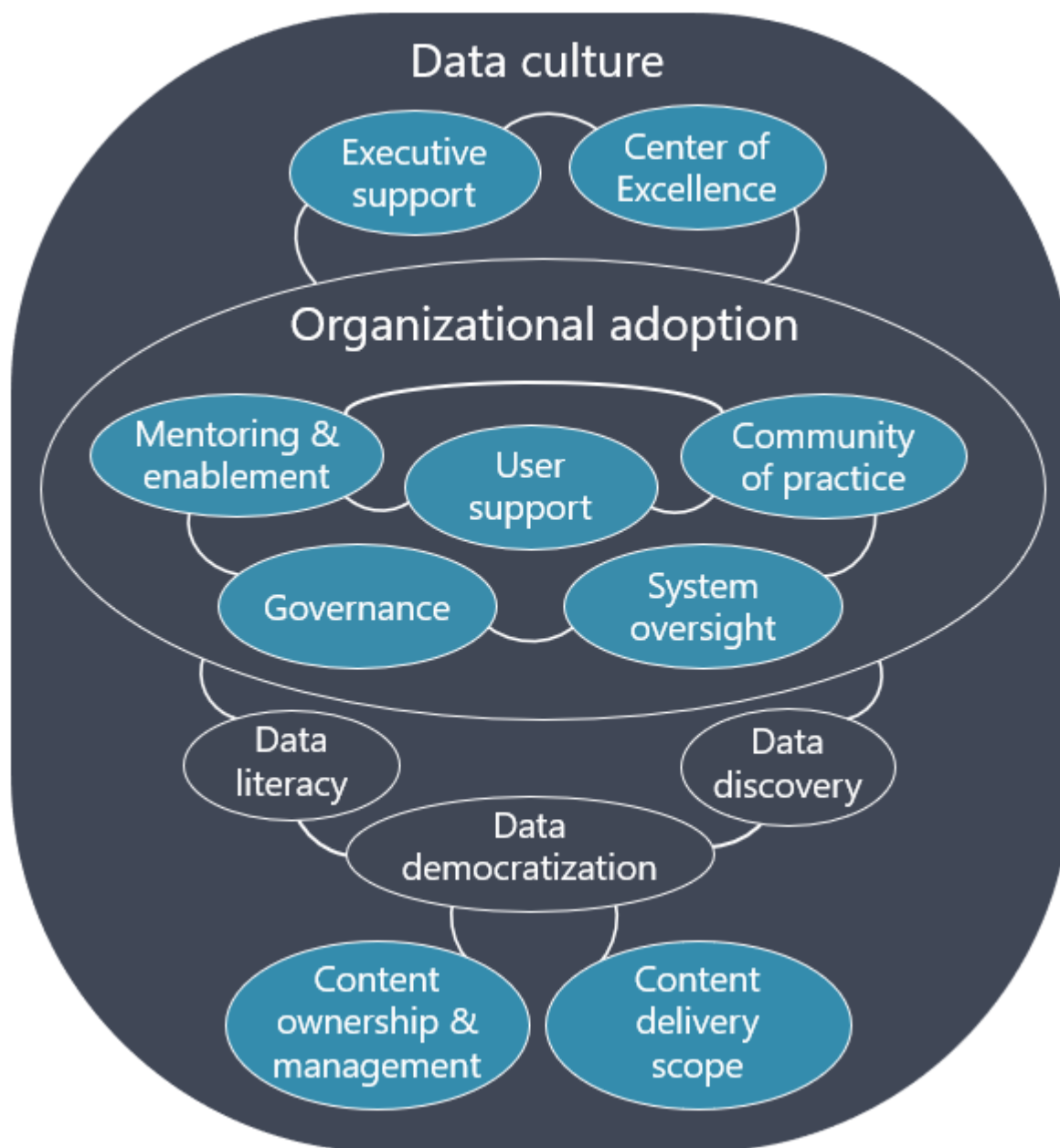
or more simply: “the exercise of decision-making and authority for data-related matters”

- ◇ <https://datagovernance.com/the-data-governance-basics/definitions-of-data-governance/>

- Data Governance is about having an enterprise data strategy
 - ◇ About *what users do with* data, not the data itself

6.A.2 Data Culture

- <https://learn.microsoft.com/en-us/power-bi/guidance/powerbi-adoption-roadmap-data-culture>



6.A.3 Key Concepts in Data Governance

- The process of managing the availability, usability, integrity, and security of an organization's data assets
- Critical to ensure that reports and dashboards are accurate, consistent, and secure
- Considerations of Data Governance
 - ◇ Data Quality
 - ◇ Data Lineage
 - ◇ Data Stewardship
 - ◇ Data Security
 - ◇ Data Governance Policies

6.A.4 Data Quality

- High quality data is essential for utility and reliability in data utilization, decision-making, and operational activities
- Quality includes accuracy, completeness, consistency, timeliness, and validity
 - ◇ Accuracy: data correctly represents the real-world object or event it describes
 - ◇ Completeness: degree to which all required data is present and accounted for
 - ◇ Consistency: data should agree with other data sets, both within and across systems
 - ◇ Timeliness: up to date, reflects current conditions
 - ◇ Validity: conforms to defined business rules, data models, or data definitions

6.A.5 Data Lineage

- Describes the origin, movement, and transformation of data as it moves through an organization's systems
- Depicts how data is used, where it came from, and what has been done to it
- Often visualized as a flowchart or a map
- Typically includes information such as the source of the data, the data's transformation, where the data is stored, and who has access to the data
- Facilitates data discovery and reuse by providing a clear understanding of the data's context

6.A.6 Data Stewardship

- The practice of overseeing an organization's data assets to ensure their proper use, protection, and compliance with policies and regulations
- A data steward is a person or a team responsible for day-to-day management of data, including planning, development, maintenance, and use of data assets
- Data stewardship includes:
 - ◇ Data Quality Management – ensuring that data is accurate, complete, consistent, and timely
 - ◇ Data Security – protecting data from unauthorized access, use, disclosure, or destruction
 - ◇ Data Privacy – ensuring compliance with regulations and policies related to data privacy
 - ◇ Data Lifecycle Management – from creation to disposal
 - ◇ Data Classification and Metadata Management – sensitivity classification and management of metadata, such as data definitions, lineage, and business rules

6.A.7 Data Security

- Protection of data from unauthorized access, use, disclosure, modification, or destruction.
- Involves all technologies, policies, procedures, and controls designed to prevent data breaches, cyber attacks, and mismanagement that can compromise data confidentiality, integrity, and availability.
 - ◇ Authentication and Authorization – typically through the use of usernames, passwords, biometrics, and other access controls.
 - ◇ Data Encryption – particularly for sensitive data
 - ◇ Network Security – firewalls, intrusion detection and prevention systems, and other security measures
 - ◇ Data Backup and Recovery – ensuring that data can be restored in the event of a disaster, system failure, or attack
 - ◇ Physical Security – may include access controls, video surveillance, and environmental controls

6.A.8 Week 11 Wrap-Up

- The six modules this week moved from advanced calculations to sharing and governance, covering the entire lifecycle of a professional BI solution
- This closes the loop on three weeks of Power BI:
 - ◇ Week 9 gave you the platform and the principles
 - ◇ Week 10 gave you the language (M and DAX) to control what the platform does
 - ◇ Week 11 gave you the advanced techniques to handle real data, the tools to share your work professionally, and the framing to think about your responsibilities as someone who shapes how an organization understands its own data
- This week also wraps up our full course of learning:
 - ◇ Excel gave you the foundations of working with spreadsheets and creating Pivot Tables and Pivot Charts
 - ◇ SQL gave you the relational database model and the ability to query databases at scale
 - ◇ Python gave you programmatic data cleaning, analysis, and the beginning of predictive modeling
 - ◇ Power BI brings those capabilities together into a connected, shareable analytics platform capable of turning raw data into organizational decisions

End of Week 11 Student Guide